EEBus Initiative e.V.
Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

EEBus Test Specification

SHIP Pairing Service

Version 1.0.0

Cologne, 2026-07-06

Terms of use for publications of EEBus Initiative e.V.

General information
The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability
EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation
The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Table of contents

Table of contents 3
List of tables 4
1 Introduction 5
1.1 References 5
1.1.1 EEBUS documents 5
1.1.2 Normative references 5
1.2 Terms and definitions 5
2 Test system definitions and prerequisites 8
2.1 Test environment 8
2.2 Test components and roles 8
2.3 Cryptographic prerequisites 8
2.4 Phrases used throughout the test case definitions 9
2.5 Parameters for test selection, execution, and conformance 11
2.6 General preconditions 14
2.6.1 Preconditions related to the DUT 14
2.6.2 Preconditions related to the test tool 16
2.6.3 Preconditions related to the tester 18
2.7 Manufacturer declarations 18
2.8 Test case conventions 19
2.8.1 Test applicability 19
2.8.2 Test execution 20
2.8.3 Handling of intermediate "Expected results" 20
2.8.4 Test repetition 20
2.8.5 Implicit requirement verifications 20
2.8.6 References between test cases 21
2.8.7 Shortening of certain 5-minute waiting periods 21
2.8.8 Shortening of certain 18-minute waiting periods 22
2.8.9 Error types for TXT records 22
3 Requirement overview and mapping to test cases 24
3.1 Requirement to test case mapping 24
3.2 Test case to requirement mapping 29
4 Test cases 31
4.1 Service Discovery (mDNS/DNS-SD) 31
4.1.1 TC_SPS_DISC_001: shippairing request announcement 1 31
4.1.2 TC_SPS_DISC_002: shippairing request announcement 2 31
4.1.3 TC_SPS_DISC_003: shippairing request deletion time 1 32
4.1.4 TC_SPS_DISC_004: shippairing request deletion time 2 32
4.1.5 TC_SPS_DISC_005: shippairing request deletion time 3 33
4.1.6 TC_SPS_DISC_006: Immediate deletion of service upon revocation of trust 34
4.1.7 TC_SPS_DISC_007: Verification of trust revocation 34
4.1.8 TC_SPS_DISC_008: Validation of TTL delay for reused instance name 35
4.1.9 TC_SPS_DISC_009: Correction of wrong shippairing requests 36
4.2 TXT record validation 37
4.2.1 TC_SPS_TXT_001: Announcement of valid TXT record 37
4.2.2 TC_SPS_TXT_002: Acceptance of rearranged TXT record 38

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.2.3 TC_SPS_TXT_003: Rejection of incomplete TXT record 39
4.2.4 TC_SPS_TXT_004: Rejection of malformed TXT record 1 39
4.2.5 TC_SPS_TXT_005: Ignore unknown keys 41
4.2.6 TC_SPS_TXT_006: Reject unsupported elliptic curve (trustCurve) 41
4.2.7 TC_SPS_TXT_007: Prevent spoofing from TXT record with duplicate trustPar 42
4.2.8 TC_SPS_TXT_008: Prevention of secret leakage in TXT record 44
4.2.9 TC_SPS_TXT_009: Rejection of malformed TXT record 2 44
4.3 Digest calculation and verification 45
4.3.1 TC_SPS_DIGEST_001: Digest verification 45
4.3.2 TC_SPS_DIGEST_002: Ring buffer properties and blocking of replay attempts 47
4.3.3 TC_SPS_DIGEST_003: Ring buffer persistence 48
4.4 Process flow and state management 50
4.4.1 TC_SPS_PROC_001: Single control unit rule 1 50
4.4.2 TC_SPS_PROC_002: Single control unit rule 2 50
4.4.3 TC_SPS_PROC_003: addCu-request process (listener) reactivation after communication loss 51
4.4.4 TC_SPS_PROC_004: Evaluation of corrected request 52
4.4.5 TC_SPS_PROC_005: Sending of parallel shippairing requests 52
4.4.6 TC_SPS_PROC_006: Sequential processing of parallel shippairing requests 53
4.4.7 TC_SPS_PROC_007: SHIP connection attempt before shippairing request 54
4.4.8 TC_SPS_PROC_008: SHIP connection acceptance before shippairing request 55
4.4.9 TC_SPS_PROC_009: addCu-request process (listener) abortion upon re-established communication 55
4.5 Trust update and management 56
4.5.1 TC_SPS_TRUST_001: devA SHIP communication after accepted shippairing request 56
4.5.2 TC_SPS_TRUST_002: Removal of trust for previous control unit 57
4.6 QR code and credential provisioning 58
4.6.1 TC_SPS_QR_001: Mandatory QR code keys for SHIP Pairing Service 58
4.7 Authentication 58
4.7.1 TC_SPS_FP_001: Rejection of invalid fingerprint of devA 58
4.7.2 TC_SPS_FP_002: Rejection of invalid fingerprint of devZ 59
5 Parallel test cases 61
5.1 TC_SPS_PARA_001: No shippairing service 61
5.2 TC_SPS_PARA_002: Only addCu-requests 61
5.3 TC_SPS_PARA_003: trustNonce variation 62
5.4 TC_SPS_PARA_004: No connection attempt to shippairing port 63
5.5 TC_SPS_PARA_005: No parallel shipparing requests to the same devA 63

List of tables

Table 1: Applicable error types 23
Table 2: Requirement to test case mapping 28
Table 3: Test case to requirement mapping 30
Table 4: Applicable errors per key 41
Table 5: Applicable errors per key 45

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

1 Introduction

This document defines a set of tests to verify the conformity of implementations with the EEBus SHIP Pairing Service Specification ([SHIPPairingService]). The tests cover the roles of both the initiator (devZ, e.g., a control unit) and the recipient (devA, e.g., a controllable device).

The test cases are designed to validate all normative requirements (marked with keyword SHALL) and recommended behaviors (SHOULD). This includes both positive scenarios and negative tests to ensure robustness and correct error handling.

1.1 References

1.1.1 EEBUS documents

[SHIP]	*Minimum*:	EEBus_SHIP_TS_Specification_v1.0.1.pdf
	Recommended:	EEBus_SHIP_TS_Specification_v1.1.0.pdf
[SHIPPairingService]	*Minimum:*	EEBus_SHIP_Pairing_Service_TS_Specification_V1.0.0.pdf
[SPINE]	*Minimum*:	EEBus_SPINE_V1.3.0_Final.zip
[SEIP]	*Minimum*:	EEBUS_TR_ShipEnhancedInstallationProcess_V1.0.0.pdf
[SRIP]	*Minimum:*	EEBUS_TS_ShipRequirementsForInstallationProcess_V1.1.0.pdf

1.1.2 Normative references

[RFC2119]	IETF RFC 2119: Key words for use in RFCs to indicate requirement levels, 1997
[RFC6762]	IETF RFC 6762: Multicast DNS, 2013
[RFC6763]	IETF RFC 6763: DNS-Based Service Discovery, 2013

1.2 Terms and definitions

addCu

A specific value for the "type" key in a shippairing request, indicating that the request originates from a Control Unit (or similar device) wishing to be added to the trusted device.

Administrator

The person or entity responsible for configuring the devices (specifically devZ) and initiating the pairing process. The administrator may perform manual tasks such as scanning QR codes or resetting trust stores.

Control unit

A device (acting as devZ) that initiates the SHIP Pairing Service to establish trust with other devices.

devA

The SHIP node that receives and evaluates the shippairing request (e.g., an energy management

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

system, charging station, or heat pump). In the context of "addCu" requests, this is the device that is being controlled or managed.

devZ
The SHIP node that initiates the pairing process by announcing the shippairing service (e.g., an MPO gateway or a control unit).

DNS-SD
Abbreviation for "Domain Name System - Service Discovery". A mechanism to browse for services on a local network.

DSO
Abbreviation for "Distribution System Operator". A DSO operates and manages the local and regional energy distribution networks.

DUT
Abbreviation for "Device Under Test". The implementation being verified by this test specification.

HMAC
Abbreviation for "Hash-based Message Authentication Code". Used to cryptographically sign the shippairing request using the devA-secret.

Installer
Person who installs, configures or operates devices. Can be an electrician hired by the customer or the DSO/MPO, or a user.

IP
Abbreviation for "Internet Protocol".

LAN
Abbreviation for "Local Area Network".

mDNS
Abbreviation for "Multicast DNS". A protocol that resolves hostnames to IP addresses within small networks that do not include a local name server.

MPO
Abbreviation for "Metering Point Operator". A MPO is a party responsible for the installation, operation, and maintenance of metering equipment for market participants, particularly in the case of smart meters.

PTR record
A DNS record type used in DNS-SD to map a service type (e.g., _shippairing._tcp) to a service instance name.

QR code
Abbreviation for "Quick Response code". A two-dimensional, electronically readable code.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

SHIP
Abbreviation for "Smart Home IP". An IP-based protocol for secure machine-to-machine communication in home automation and further domains developed by the EEBus Initiative e.V.

SKI
Abbreviation for "Subject Key Identifier". The SHA-1 value of a SHIP node's public key, used for device authentication in standard SHIP pairing.

SPINE
Abbreviation for "Smart Premises Interoperable Neutral-message Exchange". A technical specification of the EEBus Initiative e.V.

SPS
Abbreviation for "SHIP Pairing Service". The specification underlying this test specification.

SRIP
Abbreviation for "SHIP Requirements for Installation Process". A technical specification of the EEBus Initiative e.V.

SRV record
A DNS record type that specifies the hostname and port number for a service instance.

TLS
Abbreviation for "Transport Layer Security". A cryptographic protocol designed to provide communications security over a computer network.

TTL
Abbreviation for "Time to Live". A mechanism that limits the lifespan or lifetime of data in a computer or network. In DNS-SD, it dictates how long a record may be cached.

TXT record
A DNS record type used to store text-based information. In this context, it carries the payload (keys and values) of the shippairing request.

UI
Abbreviation for "User Interface". The interaction point (e.g., display, button, web interface) where a user can manually trigger actions or view status.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2 Test system definitions and prerequisites

2.1 Test environment

The tests defined in this document require a specific network environment to ensure reproducibility and to isolate the device under test (DUT) from interference.

- Local IP network: An IP network (IPv4 or IPv6) connecting the DUT and the test tool.
- Multicast support: The network infrastructure (switches, routers) must support multicast traffic to allow mDNS (multicast DNS) service discovery as per [RFC6762]. IGMP snooping or blocking filters must be disabled or configured to permit the address 224.0.0.251 (IPv4) or ff02::fb (IPv6).
- Isolation: It is recommended to execute tests in an isolated network segment to prevent interference from other SHIP devices or mDNS announcers.

2.2 Test components and roles

The SHIP Pairing Service Specification ([SHIPPairingService]) defines the roles devA and devZ. In addition, the role "monitoring" is used for cases where a test tool evaluates the fulfillment of conditions without acting as devA or devZ. The following components are defined to represent such a role or to interact with it:

- Device under test (DUT): The implementation being tested. The DUT has the role of either devA (the device to be paired) or devZ (the control unit initiating the pairing). The relevant test cases are selected upon the role of the DUT.
- Test tool: A reference implementation or dedicated software capable of simulating one or more regular or irregular counter-roles*[1] of the DUT (devA or devZ) and for monitoring network traffic on the LAN (independent of any role), including
    - orchestration of the simulated counter-roles (including monitoring of their time behaviour),
    - announcement, deletion, and verification of shippairing requests,
    - establishment and removal of (classic) SHIP trust on the test tool's SHIP instance(s),
    - establishment, interruption, and monitoring of SHIP connections.
- Tester: A human operator or an automated mechanism responsible for performing manual interactions on the DUT and the test tool required by the SHIP Pairing Service process (e.g., scanning a QR code) or related scenarios (SHIP user verify, triggering a factory reset or removing a trusted device via the user interface (UI)).

*[1]: This specification assumes that the test tool itself can fulfil these tasks. However, it is also valid to realize this in a different way (e.g., using several dedicated software tools), provided that this can be orchestrated and evaluated as required by the test descriptions.

2.3 Cryptographic prerequisites

To execute the test cases, the following cryptographic material must be available or generated:

- Certificates: X.509 certificates compliant with the SHIP specification ([SHIP]).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- Elliptic curves: The test tool MUST support at least the curve secp256r1 (NIST P-256), as this is mandatory for SHIP. Optionally, brainpoolP256r1 or brainpoolP384r1 may be used if supported by the DUT.
- Secret: The valid 128-bit integer secret of devA (devA-secret) must be known to the tester/test tool to verify the pairing process.

2.4 Phrases used throughout the test case definitions

PHR_001: Establishing a SHIP connection

A SHIP node might establish a SHIP connection actively (initiate the connection) or passively (accept a connection atempt). If a test case just requires the establishment of a connection (without using words like "actively" or "passively"), then it is not relevant which SHIP node initiated and which SHIP node accepted a SHIP connection.

PHR_002: Steps to "establish actively" or "initiate" a "SHIP communication"/"SHIP connection"; "attempt" to establish a SHIP connection

A SHIP node X tries to establish actively (i.e. initiate) a SHIP connection to a (trusted) SHIP node Y if the following steps complete successfully: SHIP node Y (as TLS server) receives a TLS handshake on its SHIP port from SHIP node X. The TLS handshake must proceed successfully until SHIP node Y receives the TLS message "Client Certificate" with the certificate of SHIP node X.

Note: This definition does not consider subsequent steps of a SHIP communication. It does also not consider if the SHIP communication is afterwards continued or aborted.

PHR_003: "SHIP communication"/"SHIP connection" is "established"

Test cases can define states where "SHIP communication" takes place or a "SHIP connection" is "established". This means that the SHIP Message Exchange state Connection Data Exchange is reached between two SHIP nodes.

Additional instructions can limit the connection/communication establishment of a test component's role to either initiate the communication or accept an incoming communication attempt.

PHR_004: "Configure the <X> ..."

Example: "Configure the DUT for SHIP and shippairing with the test tool.".

The term "configure" means any kind of possibility given to the tester to provide the SHIP node <X> (DUT or test tool) with required data. In case of the test tool, the term "configure" also means the possibility to adjust the functionality/behaviour of the test tool.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

This document does not specify the technical realisation of such a configuration. It should just be noted that esp. DUTs with the role devZ might require remote access via properly configured backend system to perform necessary changes and actions on the DUT. In accordance with section 2.2, the vendor of a DUT needs to provide the tester with an appropriate access to enable the execution of the test cases.

PHR_005: "Configure the <X> for SHIP ... with the <Y>"

Example: "Configure the DUT for SHIP ... with the test tool.".

The SHIP node <X> (DUT or test tool) is configured to trust the SHIP credentials of the SHIP node <Y> (the respective other role). Depending on the role, the credentials are the SHIP ID, SKI, SHA256 certificate fingerprint of <Y>, and the related curve.

Note: This phrase does not include trusting via shippairing credentials.

PHR_006: "Configure the <X> for... shippairing with the <Y>"

Example: "Configure the DUT for... shippairing with the test tool.".

The SHIP node <X> (DUT or test tool) is devZ and is configured with the SHIP Pairing Service credentials of the SHIP node <Y> (test tool or DUT), which is devA. These credentials are the SHIP ID, SHA256 certificate fingerprint of <Y>, the related curve, and the devA-secret.

PHR_007: "Configure the test tool ..."

Example: "Configure the test tool not to initiate any SHIP connection to trusted SHIP nodes.".

Unlike PHR_005 or PHR_006, such statements usually specify a deviation from the well-specified behaviour of SHIP or SHIP Pairing Service. Such deviations are sometimes required to evaluate a DUT behaviour in conditions that cannot be evaluated with a regular behaviour.

Frequently, such configurations are specified before the SHIP or shippairing functionality of the test tool is enabled. This means that as soon as a SHIP or shippairing functionality is enabled (see also PRE_devZConfEnabled, PRE_dutUnknownTt), the test tool begins its SHIP or shippairing functionality with the previously specified configuration.

PHR_008: "shippairing request has been deleted", "delete ... shippairing request"

For a given shippairing request (shippairing service instance), a "goodbye" announcement as specified in [RFC6762] has been sent or received.

PHR_009: "shippairing request"

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Unless stated otherwise, a shippairing request is always an "addCu-request".

PHR_010: "addCu-request"

A shippairing request where the key "type" is set to the value "addCu".

PHR_011: "enable SHIP ... of the test tool", "disable SHIP ... of the test tool"

This refers to the enabling or disabling of the test tool's own SHIP node instances (own SHIP service announcements, own SHIP server port, establishment of SHIP communication).

Preconditions or previous test steps might specify configurations that instruct to start the SHIP functionality with a properly adjusted behaviour. See PHR_007 for details.

Note: Subsequent test steps might also alter the configuration of SHIP instances (esp. to create or revoke trust into a DUT).

Independent from own SHIP node instances, the test tool might still need to monitor for SHIP or shippairing service instances or SHIP communication of the DUT. See also PHR_012.

PHR_012: "enable ... SHIP Pairing Service functionality of the test tool", "disable ... SHIP Pairing Service functionality of the test tool"

This refers to the enabling or disabling of the test tool's own SHIP Pairing Service functionality for own SHIP node instances (either listening for shippairing service announcements if test tool=devA or announcing shippairing service instances if test tool=devZ). The SHIP Pairing Service functionality must then begin with the configuration that is specified in preconditions or previous test steps.

Independent from own SHIP node instances, the test tool might still need to monitor for SHIP or shippairing service instances or SHIP communication of the DUT. See also PHR_011.

PHR_013: "monitor the <M> for at most <D> minutes"

After such a statement, an expected result is described that refers to <M>. This means that the tester/test tool must abort the test with a negative result if the expected result is not fulfilled within <D> minutes. The monitoring phase ends as soon as the expected result is fulfilled.

2.5 Parameters for test selection, execution, and conformance

This section specifies parameters (prefixed by "PAR_") that need to be set to select the relevant test cases or to execute them dependent on the given parameter. Most of the parameters denote if they apply to a DUT with the role devA or devZ.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_dutRole: DUT role

Select the role of the DUT: The DUT is either devA or devZ.

The test tool has then the other role.

PAR_spsSvcDelTime: shippairing request delete time*[1]

DUT=devZ: The vendor specifies when the DUT deletes its shippairing request after a SHIP communication is possible with the newly paired SHIP node. The duration specified by the vendor is called PAR_spsSvcDelTime_DUR. Exactly one of the following choices must be filled:

1. At most ___ (PAR_spsSvcDelTime_DUR) minutes after the first establishment of a SHIP communication with the newly paired SHIP node.
2. As soon as a SHIP communication with the newly paired SHIP node lasts for ___ (PAR_spsSvcDelTime_DUR) minutes without interruption, but not before.

The value of PAR_spsSvcDelTime_DUR is the required duration rounded up to whole minutes. The minimum value of PAR_spsSvcDelTime_DUR is 1 minute*[2]. The value of PAR_spsSvcDelTime_DUR must not exceed 15 minutes.

*[1] Note: "spsSvcDelTime" is a shorthand for "shippairing service deletion time".

*[2] Note: The rounding up of the required duration helps avoiding that test tools verify a related expected result too quickly.

PAR_typeRestrDevA: "type" restriction on devA*[1]

DUT=devA: Select if the DUT just accepts so-called "addCu-requests". This version of the test specification just permits the choice "yes".

Note: Future versions of the SHIP Pairing Service specification may reserve additional values for the TXT record field "type".

*[1] Note: "typeRestrDevA" is a shorthand for "'type' restriction on devA".

PAR_typeRestrDevZ: "type" restriction of devZ*[1]

DUT=devZ: Select if the DUT just announces so-called "addCu-requests". This version of the test specification just permits the choice "yes".

Note: Future versions of the SHIP Pairing Service specification may reserve additional values for the TXT record field "type".

*[1] Note: "typeRestrDevZ" is a shorthand for "'type' restriction of devZ".

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_spsSvcByDevA: shippairing request announcement from devA*[1]

DUT=devA: Select if the DUT announces a shippairing request. This version of the test specification just permits the choice "no".

Note: Future versions of the SHIP Pairing Service specification may define shippairing request announcements from devA.

*[1] Note: "spsSvcByDevA" is a shorthand for "shippairing service [sent] by devA".

PAR_spsSvcToDevZ: Listening to shippairing request announcements on devZ*[1]

DUT=devZ: Select if the DUT listens to shippairing requests. This version of the test specification just permits the choice "no".

Note: Future versions of the SHIP Pairing Service specification may define situations where devZ needs to listen to shippairing request announcements.

Note: There is no corresponding test case to verify PAR_spsSvcToDevZ.

*[1] Note: "spsSvcToDevZ" is a shorthand for "shippairing service [sent] to devZ".

PAR_parallelSpsSvcByDevZ: Parallel shippairing requests by devZ*[1]

DUT=devZ: Select if the DUT can send shippairing requests to more than one devA in parallel*[2]. Possible choices are "yes" (recommended), "no" (not recommended), "unspecified" (not recommended).

*[1] Note: "parallelSpsSvcByDevZ" is a shorthand for "parallel shippairing service [requests sent] by devZ".

*[2] Note: The SHIP Pairing Service specification defines how a devZ can announce shippairing requests for multiple SHIP nodes with the role devA in parallel.

PAR_spsSvcCorrection: shippairing request correction methodology of devZ*[1]

DUT=devZ: [SHIPPairingService] specifies in section 5.5 two options*[2] to announce a corrected shippairing request. Select the option that is implemented by the DUT. Possible choices are "Option 1" (recommended), "Option 2" (acceptable), "unspecified" (not recommended).

*[1] Note: "spsSvcCorrection" is a shorthand for "shippairing service correction [methodology]".

*[2] Note: "Option 1" means that a corrected shippairing request is announced using a new service instance name. "Option 2" means that a corrected shippairing request might by announced with the same service instance name as before, but the DUT must wait at least 1.5*TTL after the deletion of the (wrong) request before announcing the corrrected request.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

PAR_shipSvcByDevZ: SHIP service announcement of devZ*[1]

DUT=devZ: Select if the DUT announces its SHIP service. Possible choices are "yes" (recommended), "no" (not recommended). The choice "no" is only permissive if the DUT conforms to the German "FNN Lastenheft Steuerbox" that permits the "Steuerbox" (control box) not to announce its SHIP service. In this case, the vendor has to specify this conformance.

*[1] Note: "shipSvcByDevZ" is a shorthand for "SHIP service announcement of devZ".

PAR_eccOfDevA: Elliptic curve support of devA

DUT=devA: The vendor must specify which elliptic curves are supported for the SHIP Pairing Service functionality of devA:

- secp256r1: "supported" (the option "not supported" is not permitted in this version of the specification)
- brainpoolP256r1: "supported" or "not supported"
- brainpoolP384r1: "supported" or "not supported"

PAR_ringBufferSize: Ring buffer size

DUT=devA: The vendor must specify the size (number of entries) of the shippairing ring buffer: ___ (PAR_ringBufferSize)

PAR_ringBufferSize must be 10 or greater.

The following properties are related to PAR_ringBufferSize: The ring buffer has exactly PAR_ringBufferSize entries. The entries of the ring buffer are set in a circular order.

2.6 General preconditions

This section specifies frequently used preconditions (prefixed by "PRE_") for test components (DUT, test tool or tester). If a precondition must be fulfilled by a test component at the beginning of a test case is determined by their "applicability" and the following rules:

1. "Applicability: conditional ...":
   The precondition applies only if stated explicitly in a test case.
2. "Applicability: unconditional (every test case (with DUT=...)":
   The precondition is not mentioned explicitly in any test case. It applies anyhow for test cases with the stated role of the DUT.

2.6.1 Preconditions related to the DUT

PRE_devZShipCommCap: devZ SHIP communication capability

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

DUT=devZ: The DUT is capable of establishing a SHIP communication*[1] according to its own credentials that it uses in its shippairing requests (trustId, trustPar, trustCurve) and its SHIP TXT record (id, ski).

*[1] Note: This expresses the general SHIP communication capability (possibility to accept or initiate a SHIP connection, SHIP Message Exchange), regardless of having any trusted other SHIP node or not.

Applicability: unconditional (every test case (with DUT=devZ), unless superseded by an exception).

PRE_devZShipPort: devZ SHIP discoverability and SHIP server port

DUT=devZ: The choice in PAR_shipSvcByDevZ must be "yes". The DUT must have a SHIP server port.

Applicability: conditional (the precondition applies where stated explicitly).

PRE_devAShipCommCap: devA SHIP communication capability

DUT=devA: The DUT is capable of establishing a SHIP communication*[1] according to its own credentials stated in the QR code or label or manual or user interface.

*[1] Note: This expresses the general SHIP communication capability (possibility to accept or initiate a SHIP connection, SHIP Message Exchange), regardless of having any trusted other SHIP node or not.

Applicability: unconditional (every test case (with DUT=devA), unless superseded by an exception).

PRE_devAShipPort: devA SHIP discoverability and SHIP server port

DUT=devA: The DUT announces its SHIP service. The DUT must have a SHIP server port.

Applicability: unconditional (every test case (with DUT=devA), unless superseded by an exception).

PRE_dutUnknownTt: Test-tool-free SHIP trust store and (if DUT=devA) test-tool-free shippairing ring buffer of the DUT and listening to shippairing*[1]

DUT=devA or DUT=devZ: The DUT has no SHIP trust in any SHIP instance of the test tool, and it does not have any credentials of any SHIP instance of the test tool at the beginning of a test case. Furthermore, if DUT=devA and test tool=devZ, the DUT processes addCu-requests.

The realisation of this precondition depends on details described below*[2].

Applicability: conditional (the precondition applies where stated explicitly).

*[1] Note: "dutUnknownTt" is a shorthand for "DUT [does not trust yet the] test tool [="unknown" test tool]".

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

*[2] Details: Many test cases require that the DUT is ready for a new SHIP pairing via SHIP Pairing Service functionality with a SHIP instance of the test tool. This means that the DUT must begin with a state where it has no SHIP connection to any SHIP instance of the test tool and where it also does not trust any SHIP instance of the test tool. If the DUT has the role devA, it also means that its ring buffer must not contain entries used by the test tool. In addition, if the DUT (as devA) did accept shippairing requests before, it must be put into a state to process addCu-requests again without interference from the previous shippairing request(s) and the corresponding SHIP connection(s). Several possibilities can achieve this without forcing a factory reset of the DUT:

1. The vendor of the DUT enables the tester to easily remove all existing SHIP trusts and disconnect all SHIP connections and – if the DUT has the role devA – clean the shippairing ring buffer.
2. Alternatively, the vendor of the DUT enables the tester to remove selected SHIP trusts and disconnect the related SHIP connections and – if the DUT has the role devA – let the DUT process addCu-requests immediately (i.e. without waiting 15 minutes since the SHIP connection loss to the SHIP node with the latest accepted shippairing request). With regards to credentials that have not been removed, this approach also requires the test tool precondition PRE_ttUniqueCredTc defined in section 2.6.2.
3. Lastly, a tester might achieve this precondition even if only the removal of the trust into the latest SHIP connection is possible on the DUT. Afterwards, the SHIP connection to this node must be terminated and an additional waiting period of at least 15 minutes is required since the removal of this trust before the next test case can begin. Furthermore, this approach also requires the test tool precondition PRE_ttUniqueCredTc defined in section 2.6.2.

2.6.2 Preconditions related to the test tool

PRE_ttShipSpsOff: SHIP and shippairing disabled on the test tool*[1]

DUT=devA or DUT=devZ: The test tool has SHIP and its shippairing functionality disabled at the beginning of the test.

Applicability: unconditional (every test case (with DUT=devA or DUT=devZ), unless superseded by an exception).

*[1] Note: "ttShipSpsOff" is a shorthand for "test tool's SHIP [and] shippairing [functionality switched] off".

PRE_ttDevZShipInst: Possible SHIP instances on the devZ-test tool*[1]

DUT=devA:

1. If not stated otherwise in a test case, the test tool (as devZ) has only one SHIP instance and sends shippairing requests only for this SHIP instance to the DUT. This SHIP instance has own credentials (SHIP ID, certificate(s)).
2. If stated in a test case, the test tool will be reconfigured from one SHIP instance to another SHIP instance (resulting in different credentials). shippairing requests are then sent for the new SHIP instance.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3. If stated in a test case, the test tool will have more than one SHIP instance, each with own credentials and each (potentially) sending shippairing requests for the respective instance.

This precondition does not state if SHIP or shippairing is enabled on the test tool.

Applicability: unconditional (every test case (with DUT=devA), unless superseded by an exception).

*[1] Note: "ttDevZShipInst" is a shorthand for "test tool [=] devZ: [possible] SHIP instances".

PRE_ttDevAShipInst: Possible SHIP instances on the devA-test tool*[1]

DUT=devZ:

1. If not stated otherwise in a test case, the test tool (as devA) has only one SHIP instance and evaluates shippairing requests only for this SHIP instance to the DUT. This SHIP instance has own credentials (SHIP ID, certificate(s)).
2. If stated in a test case, the test tool will be reconfigured from one SHIP instance to another SHIP instance (resulting in different credentials). shippairing requests are then evaluated for the new SHIP instance.
3. If stated in a test case, the test tool will have more than one SHIP instance, each with own credentials and each (potentially) evaluating shippairing requests for the respective instance.

This precondition does not state if SHIP or shippairing is enabled on the test tool.

Applicability: unconditional (every test case (with DUT=devA), unless superseded by an exception).

*[1] Note: "ttDevAShipInst" is a shorthand for "test tool [=] devA: [possible] SHIP instances".

PRE_ttUnknownDut: DUT-free SHIP trust store and (if DUT=devZ) empty shippairing ring buffer of the test tool*[1]

DUT=devA or DUT=devZ: If not stated otherwise, the test tool has no SHIP trust in the DUT, and it does not have any credentials of the DUT at the beginning of a test case. Furthermore, if DUT=devZ and test tool=devA, the test tool has an empty shippairing ring buffer.

Applicability: conditional (the precondition applies where stated explicitly).

*[1] Note: "ttUnknownDut" is a shorthand for "test tool [does not trust yet the] DUT [="unknown" DUT]".

PRE_ttUniqueCredTc: Unique test tool credentials per test case executions*[1]

DUT=devA or DUT=devZ: Credentials of the test tool's SHIP instances are unique for each test case execution*[2].

Applicability: conditional (the precondition applies where stated explicitly)*[3].

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

*[1] Note: "ttUniqueCredTc" is a shorthand for "test tool [has] unique credentials per test case".

[2] Note: If a test case execution begins, the test tool must ensure that its SHIP instances will not reuse any credentials (SHIP IDs, certificates, secrets) that have been used any time in the past with the DUT[4]. This does not only hold true between test case "X" and test case "Y". It also holds true if a test case "X" is executed again (because it must be iterated according to a requirement or because it must be repeated for other reasons).

*[3] Note: See esp. the DUT precondition "PRE_dutUnknownTt" in section 2.6.1.

*[4] Note: This does not refer to the reference between test cases as described in section 2.8.6: There, a test case "X" reuses preconditions, test steps, and expected result of another test case "Y". Nevertheless, all credentials are associated with test case "X" only. But the test tool's credentials of test case "X" must differ from those of the – independently executed – test case "Y".

2.6.3 Preconditions related to the tester

PRE_credAvail: Credential availability*[1]

DUT=devA: The following data of the DUT is available to the tester:

1. The DUT's SHIP ID.
2. For each supported curve: The curve name, the SKI, and the SHA256 fingerprint of the DUT's certificate.
3. The devA-secret.

Applicability: unconditional (every test case (with DUT=devA), unless superseded by an exception).

*[1] Note: "credAvail" is a shorthand for "credential availability".

PRE_devZConfEnabled: Tester enabled for devZ configurations*[1]

DUT=devZ: The tester must be enabled to perform tasks of the administrator of devZ (configure devZ to trust devA or revoke the trust in devA (including abortion of a SHIP communication with devA), configure devZ to create a proper shippairing request or to withdraw it).

Applicability: unconditional (every test case (with DUT=devZ), unless superseded by an exception).

*[1] Note: "devZConfEnabled" is a shorthand for "devZ configuration [possibility] enabled [for the tester]".

2.7 Manufacturer declarations

Some requirements defined in the SHIP Pairing Service Specification ([SHIPPairingService]) refer to internal device behaviours, algorithmic implementations (such as random number generation), or the structural separation of cryptographic secrets. Since these internal mechanisms cannot be exhaustively verified through external blackbox testing, the vendor must formally declare and

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

guarantee compliance with these specific requirements (prefixed by "DEC_") before the test execution begins.

DEC_devASecretRandomness: Randomness and uniqueness of devA-secret

Applicability: DUT=devA

Description: The vendor declares that the devA-secret is generated using a proper random number generator to create a practically unique secret for each devA instance, preventing any two different devA instances from having the same secret, in accordance with [SHIPPairingService] section 6.2 (or 6.3 respectively).

DEC_trustNonceRandomness: Randomness of trustNonce

Applicability: DUT=devZ

Description: The vendor declares that for each new SHIP Pairing Service process, a proper random number generator is used to create a practically unique value for the key trustNonce, in accordance with [SHIPPairingService] section 6.2 (or 6.3 respectively).

DEC_secretNonExposure: Prevention of secret leakage in TXT record

Applicability: DUT=devZ

Description: The vendor declares that the devA-secret is strictly used only for the digest calculation. It is not used in any way to create the value of trustNonce, nor is the secret (or parts of it) added to any other TXT record keys or additional keys, in strict accordance with [SHIPPairingService] section 5.4.
Note: While test case TC_SPS_TXT_008 performs a basic substring check on the TXT record, this declaration ensures structural and algorithmic compliance.

2.8 Test case conventions

This section defines specific terms and procedural rules used within the test case definitions of this document.

2.8.1 Test applicability

This field defines the conditions under which a test case is relevant for a specific DUT. The applicability is usually based on the role of the DUT (devA or devZ) or specific parameter declarations made by the vendor (see section 2.5, e.g., PAR_spsSvcCorrection or PAR_parallelSpsSvcByDevZ). If the conditions defined in "Test applicability" are not met, the test case is considered "not applicable" and shall be skipped.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2.8.2 Test execution

This field specifies if a test case consists of multiple distinct execution runs (e.g., with different configurations or parameters). It defines the specific settings for each run. Unless stated otherwise, all defined test runs must be completed successfully to reach a positive overall test verdict for the test case. Preconditions apply to each individual test run.

2.8.3 Handling of intermediate "Expected results"

Some test cases define an "Expected result" immediately following a specific test step, before the test case description ends. These intermediate results serve as pass/fail gateways:

- Success: If the observed behaviour matches the expected result, the test execution continues with the next test step.
- Failure: If the observed behaviour does not match the expected result, the test case is immediately terminated and assessed as failed. The subsequent test steps are not executed.

2.8.4 Test repetition

Most test cases yield a deterministic result and require only a single execution. However, some test cases verify behaviours subject to timing variances, race conditions or network latencies (e.g., verifying a connection attempt before a service announcement). For such cases, the test case defines a "Test repetition" rule (e.g., "The test passes if at least 1 of up to 20 iterations yield the expected result").

If a "Test repetition" rule exists in a test case, the following applies:

- The tester repeats the full test case up to the specified maximum number of iterations.
- The clause can specifiy conditions to abort the iterations.
- Each repetition must fulfill the preconditions.
- The clause specifies conditions on the iterations to assess the test case as failed or passed.

Otherwise, the test case shall be executed exactly once.

2.8.5 Implicit requirement verifications

The EEBus SHIP Pairing Service Specification makes use of mDNS/DNS-SD service instances with "_shippairing._tcp" for <Service> and "local." for <Domain> (see [SHIPPairingService] chapter 5). The specification also recommends the form "<SHIP-Instance><Delimiter><Counter>" for the <Instance> part. Such requirements are not covered explicitly by test cases. Instead, a test tool uses these requirements implicitly to search for such services or to announce such services.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2.8.6 References between test cases

Some test cases reuse definitions of other test cases. This means that a test case "X" might require in its preconditions the preconditions of another test case "Y". The first test step of test case "X" then requires executing the test steps of test case "Y" and also requires achieving the expected result of test case "Y". Afterwards, test case "X" continues with the very own test steps.

This concept serves to reduce the content of test case "X" to what is essential for its test objective. Any problems in the execution of test steps or deviations from expected results are associated with test case "X" only. This also holds true during the first test step of "X", i.e. if there is a problem with the execution of a test step of "Y" or a deviation from the respective expected result.

Please note that such a referenced test case "Y" might also reference preconditions and test steps of another test case "Z".

2.8.7 Shortening of certain 5-minute waiting periods

This section applies only where referenced explicitly.

Several test cases where the DUT has the role devA contain test steps that require a waiting time of 5 minutes:

> "Monitor ... for 5 minutes (see section 2.8.7 to shorten this period)"

These waiting periods are required esp. where the rejection of a shippairing request is to be verified. As this can lead to a long duration in some test cases, a possibility to shorten the waiting time of 5 minutes can be applied. The following requirements must be fulfilled for this possibility:

- The DUT can log each received shippairing request and its evaluation result (accepted/rejected) with the timestamp. This log is made available to the tester.
- The test tool can log its announced shippairing requests with their timestamp.

If these requirements are fulfilled, the tester can configure the test tool to

- shorten the properly marked waiting time to, e.g., 30 seconds,
- and verify if a SHIP connection is still rejected at the end of this waiting period.

After the test case is completed, the tester can compare the DUT's log with the test tool's log and verify if the DUT had sufficient time to receive and evaluate the shippairing requests. If this is not the case the test case must be repeated with a higher waiting period.

The log of the DUT must contain the following information regarding the received shippairing request:

- The timestamp.
- The service instance name.
- The value of the TXT record key "digest".
- The value of the TXT record key "alg".

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

The log of the test tool must contain the same information regarding the announced shippairing request. Additionally, the logged entry must contain the following information regarding the test case:

- The test case name and test step including iteration counters (if applicable).
- An indication if the digest was intentionally valid or invalid.

2.8.8 Shortening of certain 18-minute waiting periods

This section applies only where referenced explicitly.

Several test cases where the DUT has the role devA contain test steps that require a waiting time of 18 minutes:

"Wait for 18 minutes (see section 2.8.8 to shorten this period)"

These waiting periods are required to reactivate the DUT's processing of addCu-requests after a communication loss with the current devZ (15 minutes required by [SHIPPairingService] and additional 3 minutes as safety margin for testing). In test steps that refer to this section this waiting time is not required for the respective test case's objective (it is just required to conform to [SHIPPairingService]). In such cases a possibility to shorten the waiting time of 18 minutes can be applied. The following requirement must be fulfilled for this possibility:

- The vendor of the DUT enables the tester to immediately reactivate the DUT's processing of addCu-requests. This option is subject to the following restrictions:
  - This option must only be possible if the DUT currently has no SHIP communication with its current devZ.
  - The immediate reactivation to process addCu-requests is no permanent configuration. This means that the tester must enable this option manually every time the waiting time to reactivate the processing of addCu-requests has to be skipped.

Note: In some test cases the DUT must prove that it waits for 15 minutes before it reactivates the processing of addCu-requests. In the respective test steps the waiting time of 18 minutes must not be shortened. Consequently, such test steps do not refer to this section.

2.8.9 Error types for TXT records

This section applies only where referenced explicitly.

In several test cases the test tool creates TXT records with specific errors for the key name or key value. Table 1 specifies error types with their ID and technical details on the error that is to be applied.

Error type ID	Error to be applied
E01	Omit the last letter of the key name.
E02	Append a single letter (from 'a' to 'z') to the key name.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

E03	Set the value to any of the following strings (without quotation marks): "0", "2", "3, "4", "5", "6", "7", "8", "9", "10", "11"
E04	Set the value to any of the following strings (without quotation marks): "fpsha256", "fpSha2560", "fpSha25", "fpSha512"
E05	Curtail the value to any length from 0 to 63 characters.
E06	Curtail the value to any length from 0 to 31 characters.
E07	Append any of the following strings (without quotation marks) to the value: "G", "+", "A", "9B"
E08	Convert the value to lowercase. Note: The possibility of absence of any letter in the value is neglected.
E09	If the original value is "secp256r1": Set the value to any of the following strings (without quotation marks): "secp256R1", "secp256r11", "P-256", "NIST P-256". If the original value is "brainpoolP256r1": Set the value to any of the following strings (without quotation marks): "brainpoolp256r1", "brainpoolP256r12", "brainpoolP256". If the original value is "brainpoolP384r1": Set the value to any of the following strings (without quotation marks): "brainpoolp384r1", "brainpoolP384r13", "brainpoolP384r".
E10	Set the value to any of the following strings (without quotation marks): "addcu", "addC", "addCuj"
E11	Set the value to any of the following strings (without quotation marks): "hmacsha256", "hmacSha2561", "hmacSha25", "hmacSha512"
E12	Set the value to an empty string.
E13	Change the letter case of any letter of the value. Note: The possibility of absence of any letter in the value is neglected.
E14	Append any of the following strings (without quotation marks) to the value: "00", "8C", "FF".
E15	Curtail the value to any even length from 0 to 30 characters*[1].

Table 1: Applicable error types

*[1] Note: The length must be even if an algorithm requires a byte array (for example for the key K for the digest calculation as specified in [SHIPPairingService], section 7.3). This corresponds with an even number of hexadecimal characters.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3 Requirement overview and mapping to test cases

3.1 Requirement to test case mapping

This section provides a mapping from the requirements defined in the SHIP Pairing Service Specification ([SHIPPairingService]) to test cases in this document. Table 2 focuses on the primary test case(s) that test a specific requirement. This means that a requirement might implicitely be tested by further test cases without being mentioned in the table below.

Req-ID	Source	Requirement (excerpt)	Test case ID Declaration ID
SPS-REQ-4-001	SPS 4.2	Before devZ sends its shippairing service instance for devA, devZ SHALL trust devA with regards to SHIP connections to and from devA [...].	TC_SPS_PROC_007 (4.4.7) TC_SPS_PROC_008 (4.4.8)
SPS-REQ-4-002	SPS 4.2	When configured properly, devZ SHALL create a proper shippairing request according to chapter 8.	TC_SPS_TXT_001 (4.2.1) TC_SPS_DISC_002 (4.1.2)
SPS-REQ-4-003	SPS 4.2	As soon as devZ successfully completed a SHIP connection with devA [...], devZ SHOULD continue with the announcement of its shippairing service instance for devA until devZ had an uninterrupted SHIP connection with devA for 15 minutes [...]. Afterwards, devZ SHALL delete its shippairing service instance for devA [...].	TC_SPS_DISC_003 (4.1.3) TC_SPS_DISC_004 (4.1.4) TC_SPS_DISC_005 (4.1.5)
SPS-REQ-4-004	SPS 4.2	If the administrator of devZ configures devZ to untrust devA while devZ still announces the shippairing service instance for devA [...], devZ SHALL delete its shippairing service instance for devA immediately.	TC_SPS_DISC_006 (4.1.6) TC_SPS_DISC_007 (4.1.7)
SPS-REQ-4-005	SPS 4.2	[...] if devA's functionality to receive and evaluate shippairing requests is activated, devA SHALL listen to/search for shippairing requests and evaluate them according to chapter 9.	TC_SPS_TRUST_001 (4.5.1) TC_SPS_TXT_002 (4.2.2) TC_SPS_TXT_005 (4.2.5) TC_SPS_DIGEST_001 (4.3.1) TC_SPS_PROC_004 (4.4.4)
SPS-REQ-4-006	SPS 4.2	If devA was not able to successfully evaluate and accept a shippairing request according to chapter 9, devA SHALL NOT continue with the subsequent steps. Instead, devA SHALL ignore the current announcement of the shippairing service instance.	TC_SPS_TXT_003 (4.2.3) TC_SPS_TXT_004 (4.2.4) TC_SPS_TXT_006 (4.2.6) TC_SPS_TXT_007 (4.2.7) TC_SPS_TXT_009 (4.2.9)
SPS-REQ-4-007	SPS 4.2	If devA successfully evaluated and accepted an addCu-request, devA SHALL deactivate its processing of addCu-requests […]. Then, dev A SHALL trust the SHIP node […]. devA SHALL then conform to the SHIP specification to establish a SHIP connection with devZ […].	TC_SPS_PROC_001 (4.4.1) TC_SPS_TRUST_001 (4.5.1)

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

SPS-REQ-4-008	SPS 4.2	devA SHALL conform to section 4.3 to deactivate or to reactivate processing addCu-requests.	TC_SPS_PROC_001 (4.4.1) TC_SPS_PROC_002 (4.4.2) TC_SPS_PROC_003 (4.4.3)
SPS-REQ-4-009	SPS 4.3	[...] devA SHALL stop processing addCu-requests once it accepted an addCu-request and as long as it intends to continue this pairing with devZ.	TC_SPS_PROC_001 (4.4.1) TC_SPS_PROC_002 (4.4.2)
SPS-REQ-4-010	SPS 4.3	If devA receives and successfully evaluates a new valid addCu-request on the new devZ2, devA SHALL untrust the former devZ1, SHALL trust the new devZ2, and SHALL deactivate processing addCu-requests.	TC_SPS_PROC_003 (4.4.3) TC_SPS_TRUST_002 (4.5.2)
SPS-REQ-4-011	SPS 4.3	If otherwise a SHIP connection with devZ1 is suddenly possible again, devA SHALL deactivate processing addCu-requests.	TC_SPS_PROC_009 (4.4.9)
SPS-REQ-4-012	SPS 4.4	[...] it is permissive for devZ to send the required shippairing requests [to both nodes] in parallel. [...] devZ SHALL announce these requests with individual service instance names [...].	TC_SPS_PROC_005 (4.4.5)
SPS-REQ-4-013	SPS 4.4	devZ SHALL NOT send parallel (different) shippairing requests to a particular devA.	TC_SPS_DISC_009 (4.1.9) TC_SPS_PARA_005 (5.5)
SPS-REQ-4-014	SPS 4.1	[...] each shippairing request is a so-called "addCu-request". An addCu-request is determined by the value "addCu" in the TXT record key "type" [...].	TC_SPS_PARA_002 (5.2)
SPS-REQ-4-015	SPS 4.1	[...] devA does not announce a shippairing request (to devZ) [...]	TC_SPS_PARA_001 (5.1)
SPS-REQ-4-016	SPS 4.3	An automatic reactivation for processing addCu-requests requires that the following condition is fulfilled: [...] devA is not able to perform SHIP Message Exchange with the paired devZ1 since at least 15 minutes (devA is running continuously in this period).	TC_SPS_PROC_002 (4.4.2)
SPS-REQ-5-001	SPS 5.2	[...] a shippairing service SHALL use "_shippairing._tcp" [...] for <Service>, and "local." [...] for <Domain>.	Implicitly tested (see section 2.8.5), for example in TC_SPS_DISC_001 (4.1.1)
SPS-REQ-5-003	SPS 5.2	[...] the shippairing <Instance> SHOULD have the form <SHIP-Instance><Delimiter><Counter> where <Counter> is a string representing a positive integer value [...] and <Delimiter> is a character to separate <SHIP-Instance> from <Counter>.	Implicitly tested (see section 2.8.5), for example in TC_SPS_PROC_005 (4.4.5)
SPS-REQ-5-007	SPS 5.3	devA SHALL NOT send data to a port of a shippairing SRV record.	TC_SPS_PARA_004 (5.4)
SPS-REQ-5-008	SPS 5.4	devZ SHALL provide for an active shippairing service instance a TXT record according to Table 1.	TC_SPS_TXT_001 (4.2.1)

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

SPS-REQ-5-009	SPS 5.4	The key "txtvers" SHALL be the first key in the TXT record.	TC_SPS_TXT_001 (4.2.1) TC_SPS_TXT_002 (4.2.3)
SPS-REQ-5-010	SPS 5.4	The values of the keys "forId" and "trustId" SHALL be set and interpreted in UTF-8 format.	TC_SPS_TXT_001 (4.2.1) TC_SPS_TXT_003 (4.2.3)
SPS-REQ-5-011	SPS 5.4	The values SHALL NOT be empty.	TC_SPS_TXT_001 (4.2.1) TC_SPS_TXT_004 (4.2.4) TC_SPS_TXT_009 (4.2.9)
SPS-REQ-5-012	SPS 5.4	For each new shippairing request […], devZ SHALL create a new random number as value for the key "trustNonce" according to section 6.2.	TC_SPS_PARA_003 (5.3) DEC_trustNonceRandomness (see section 2.7)
SPS-REQ-5-013	SPS 5.4	devA SHALL ignore keys that are not mentioned in Table 1.	TC_SPS_TXT_005 (4.2.5)
SPS-REQ-5-014	SPS 5.4	devA SHALL consider a TXT record being invalid if a mandatory key is missing […].	TC_SPS_TXT_003 (4.2.3)
SPS-REQ-5-015	SPS 5.4	devA SHALL consider a TXT record being invalid […], if a specified key has a value that does not conform to this specification […].	TC_SPS_TXT_004 (4.2.4) TC_SPS_TXT_009 (4.2.9)
SPS-REQ-5-016	SPS 5.4	devA SHALL consider a TXT record being invalid if the first key is not "txtvers".	TC_SPS_TXT_003 (4.2.3)
SPS-REQ-5-017	SPS 5.4	devA SHALL consider a TXT record being invalid if the value of the key "trustCurve" denotes an elliptic curve that devA does not support […].	TC_SPS_TXT_006 (4.2.6)
SPS-REQ-5-018	SPS 5.4	devZ SHALL NOT use the devA-secret in any way to create the value of "trustNonce".	TC_SPS_TXT_008 (4.2.8) DEC_secretNonExposure (see section 2.7)
SPS-REQ-5-019	SPS 5.4	devZ SHALL NOT add the devA-secret or parts of it in any way to any of the other TXT record keys.	TC_SPS_TXT_008 (4.2.8) DEC_secretNonExposure (see section 2.7)
SPS-REQ-5-020	SPS 5.4	devZ SHALL NOT put any additional key into the TXT record that contains the devA-secret in any way.	TC_SPS_TXT_008 (4.2.8) DEC_secretNonExposure (see section 2.7)
SPS-REQ-5-022	SPS 5.5	If devZ sent a shippairing request with "static" keys that are not valid anymore and need to be replaced, devZ SHALL delete the current (invalid) shippairing request first [...].	TC_SPS_DISC_008 (4.1.8)
SPS-REQ-5-023	SPS 5.5	After the deletion of the old shippairing request, devZ SHALL wait for at least 1.5*TTL_old before it creates a new shippairing request [based upon "option 2" using the same service instance name as the previous one].	TC_SPS_DISC_008 (4.1.8)
SPS-REQ-6-003	SPS 6.2	For each of these certificates, the corresponding fingerprint SHALL be calculated as the SHA-256 value […] of the certificate in the DER format [...].	TC_SPS_TXT_001 (4.2.1)
SPS-REQ-6-004	SPS 6.2	devA SHALL have a secret value for SHIP Pairing Service.	TC_SPS_TRUST_001 (4.5.1) TC_SPS_QR_001 (4.6.1)

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

SPS-REQ-6-005	SPS 6.2	The value [of the secret] SHALL be an integer value in the range [0, 2^{128}-1].	TC_SPS_TXT_001 (4.2.1) TC_SPS_TXT_004 (4.2.4) TC_SPS_TXT_009 (4.2.9) TC_SPS_QR_001 (4.6.1)
SPS-REQ-6-006	SPS 6.2	Per devA, a random number generator […] SHALL be used to create a practically unique secret for devA.	DEC_devASecretRandomness (2.7)
SPS-REQ-6-007	SPS 6.2	[Vendors] SHALL do their best effort to prevent that any two different devA instances (SHIP nodes) they produce have the same secret.	DEC_devASecretRandomness (2.7)
SPS-REQ-6-010	SPS 6.2	trustNonce (devZ-nonce): [...][The] value SHALL be an integer value in the range [0, 2^{128}-1].	TC_SPS_TXT_001 (4.2.1)
SPS-REQ-6-011	SPS 6.2	Per devA as well as per new SHIP Pairing Service process with a given devA, a random number generator […] SHALL be used to create a practically unique value of "trustNonce".	TC_SPS_PARA_003 (5.3) DEC_trustNonceRandomness (see section 2.7)
SPS-REQ-6-012	SPS 6.2	devZ SHALL NOT reuse the value of trustNonce of an earlier successful shipPairing request to devA for a later re-execution of a SHIP Pairing Service process with devA.	TC_SPS_PARA_003 (5.3) DEC_trustNonceRandomness (see section 2.7)
SPS-REQ-7-002	SPS 7.4	[Create (temporary) message M:] It is not permissive to deviate from the specified construction of M. This also means that it is not permissive to alter the order of keys (and values) or to add or remove entries.	TC_SPS_TXT_002 (4.2.2)
SPS-REQ-9-001	SPS 9	devA SHALL NOT evaluate two or more shippairing requests in parallel.	TC_SPS_PROC_006 (4.4.6)
SPS-REQ-9-002	SPS 9	devA SHALL continue with the evaluation of a TXT record [...] only if all of the following conditions are fulfilled: • The key "txtvers" is the first key in the TXT record and has the value as specified in section 5.4. • The key "forId" is present and set to devA's SHIP ID. • The key "parType" is present and set to a value supported by devA. • The key "forPar" is present and [...] uniquely identifies a SHIP certificate of devA. • The value of the key "trustCurve" is present and set to a value supported by devA. • [...] the TXT record key "type" is set to "addCu" [...]. • The TXT record fulfils requirements of section 5.4 in general (i.e. all	TC_SPS_TXT_002 (4.2.2) TC_SPS_TXT_003 (4.2.3) TC_SPS_TXT_004 (4.2.4) TC_SPS_TXT_006 (4.2.6) TC_SPS_TXT_009 (4.2.9)

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

		mandatory keys are present and contain permitted values).	
SPS-REQ-9-003	SPS 9	If the ring buffer already contains an entry with this pair [{ <value of key "alg">, <value of key "digest"> }], then devA SHALL abort the evaluation of this shippairing request.	TC_SPS_DIGEST_002 (4.3.2)
SPS-REQ-9-004	SPS 9	devA SHALL abort the evaluation of this TXT record [if the calculated digest is not identical to the (converted) value of the TXT record key "digest"].	TC_SPS_DIGEST_001 (4.3.1)
SPS-REQ-9-005	SPS 9	devA SHALL add "new-pair" to the ring buffer as described in section 11.3.	TC_SPS_DIGEST_002 (4.3.2) TC_SPS_DIGEST_003 (4.3.3)
SPS-REQ-10-001	SPS 10.2	For SHIP Pairing Service, the fingerprint of the certificate received during the TLS handshake will be calculated [...] and verified with the expected fingerprint of the shippairing request. If the fingerprints match, this SHALL be seen like a successful trust in an SKI from the perspective of the SHIP specification.	TC_SPS_FP_001 (4.7.1) TC_SPS_FP_002 (4.7.2)
SPS-REQ-10-002	SPS 10.3	[...]devA SHALL only trust the current devZ with regards to SHIP Pairing Service trust establishment.	TC_SPS_TRUST_002 (4.5.2)
SPS-REQ-11-001	SPS 11.2	devA SHALL have a ring buffer "digest-ring-buffer" with at least 10 entries.	TC_SPS_DIGEST_002 (4.3.2)
SPS-REQ-11-002	SPS 11.2	devA SHALL store [the] "digest-ring-buffer" [...] persistently.	TC_SPS_DIGEST_003 (4.3.3)
SPS-REQ-12-001	SPS 12	If a QR code contains the SKI of a given curve, the QR code SHALL also contain the related fingerprint.	TC_SPS_QR_001 (4.6.1)
SPS-REQ-12-002	SPS 12	[...] the devA-secret SHALL be part of the QR code […].	TC_SPS_QR_001 (4.6.1)

Table 2: Requirement to test case mapping

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3.2 Test case to requirement mapping

This section provides a reverse mapping from the test cases defined in this document to the requirements of the SHIP Pairing Service Specification ([SHIPPairingService]). This list allows to identify which specific requirements are verified by a given test case. It assists in impact analysis to determine which specification clauses are violated in the event of a specific test failure.

Test case ID	Section	DUT	Req-ID	Source
TC_SPS_DISC_001	4.1.1	devZ	SPS-REQ-5-001	SPS 5.2
TC_SPS_DISC_002	4.1.2	devZ	SPS-REQ-4-002	SPS 4.2
TC_SPS_DISC_003	4.1.3	devZ	SPS-REQ-4-003	SPS 4.2
TC_SPS_DISC_004	4.1.4	devZ	SPS-REQ-4-003	SPS 4.2
TC_SPS_DISC_005	4.1.5	devZ	SPS-REQ-4-003	SPS 4.2
TC_SPS_DISC_006	4.1.6	devZ	SPS-REQ-4-004	SPS 4.2
TC_SPS_DISC_007	4.1.7	devZ	SPS-REQ-4-004	SPS 4.2
TC_SPS_DISC_008	4.1.8	devZ	SPS-REQ-5-022	SPS 5.5
			SPS-REQ-5-023	SPS 5.5
TC_SPS_DISC_009	4.1.9	devZ	SPS-REQ-4-013	SPS 4.4
TC_SPS_TXT_001	4.2.1	devZ	SPS-REQ-4-002	SPS 4.2
			SPS-REQ-5-008	SPS 5.4
			SPS-REQ-5-009	SPS 5.4
			SPS-REQ-5-010	SPS 5.4
			SPS-REQ-5-011	SPS 5.4
			SPS-REQ-6-003	SPS 6.2
			SPS-REQ-6-005	SPS 6.2
			SPS-REQ-6-010	SPS 6.2
TC_SPS_TXT_002	4.2.2	devA	SPS-REQ-4-005	SPS 4.2
			SPS-REQ-5-009	SPS 5.4
			SPS-REQ-7-002	SPS 7.4
			SPS-REQ-9-002	SPS 9
TC_SPS_TXT_003	4.2.3	devA	SPS-REQ-4-006	SPS 4.2
			SPS-REQ-5-010	SPS 5.4
			SPS-REQ-5-014	SPS 5.4
			SPS-REQ-5-016	SPS 5.4
			SPS-REQ-9-002	SPS 9
TC_SPS_TXT_004	4.2.4	devA	SPS-REQ-4-006	SPS 4.2
			SPS-REQ-5-011	SPS 5.4
			SPS-REQ-5-015	SPS 5.4
			SPS-REQ-6-005	SPS 6.2
			SPS-REQ-9-002	SPS 9
TC_SPS_TXT_005	4.2.5	devA	SPS-REQ-4-005	SPS 4.2
			SPS-REQ-5-013	SPS 5.4
TC_SPS_TXT_006	4.2.6	devA	SPS-REQ-4-006	SPS 4.2
			SPS-REQ-5-017	SPS 5.4
			SPS-REQ-9-002	SPS 9
TC_SPS_TXT_007	4.2.7	devA	SPS-REQ-4-006	SPS 4.2
TC_SPS_TXT_008	4.2.8	devZ	SPS-REQ-5-018	SPS 5.4
			SPS-REQ-5-019	SPS 5.4
			SPS-REQ-5-020	SPS 5.4
TC_SPS_TXT_009	4.2.9	devA	SPS-REQ-4-006	SPS 4.2
			SPS-REQ-5-011	SPS 5.4

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

			SPS-REQ-5-015	SPS 5.4
			SPS-REQ-6-005	SPS 6.2
			SPS-REQ-9-002	SPS 9
TC_SPS_DIGEST_001	4.3.1	devA	SPS-REQ-4-005	SPS 4.2
			SPS-REQ-9-004	SPS 9
TC_SPS_DIGEST_002	4.3.2	devA	SPS-REQ-9-003	SPS 9
			SPS-REQ-9-005	SPS 9
			SPS-REQ-11-001	SPS 11.2
TC_SPS_DIGEST_003	4.3.3	devA	SPS-REQ-9-005	SPS 9
			SPS-REQ-11-002	SPS 11.2
TC_SPS_PROC_001	4.4.1	devA	SPS-REQ-4-007	SPS 4.2
			SPS-REQ-4-008	SPS 4.2
			SPS-REQ-4-009	SPS 4.3
TC_SPS_PROC_002	4.4.2	devA	SPS-REQ-4-008	SPS 4.2
			SPS-REQ-4-009	SPS 4.3
			SPS-REQ-4-016	SPS 4.3
TC_SPS_PROC_003	4.4.3	devA	SPS-REQ-4-008	SPS 4.2
			SPS-REQ-4-010	SPS 4.3
TC_SPS_PROC_004	4.4.4	devA	SPS-REQ-4-005	SPS 4.2
TC_SPS_PROC_005	4.4.5	devZ	SPS-REQ-4-012	SPS 4.4
			SPS-REQ-5-003	SPS 5.2
TC_SPS_PROC_006	4.4.6	devA	SPS-REQ-9-001	SPS 9
TC_SPS_PROC_007	4.4.7	devZ	SPS-REQ-4-001	SPS 4.2
TC_SPS_PROC_008	4.4.8	devZ	SPS-REQ-4-001	SPS 4.2
TC_SPS_PROC_009	4.4.9	devA	SPS-REQ-4-011	SPS 4.3
TC_SPS_TRUST_001	4.5.1	devA	SPS-REQ-4-005	SPS 4.2
			SPS-REQ-4-007	SPS 4.2
			SPS-REQ-6-004	SPS 6.2
TC_SPS_TRUST_002	4.5.2	devA	SPS-REQ-4-010	SPS 4.3
			SPS-REQ-10-002	SPS 10.3
TC_SPS_QR_001	4.6.1	devA	SPS-REQ-6-004	SPS 6.2
			SPS-REQ-6-005	SPS 6.2
			SPS-REQ-12-001	SPS 12
			SPS-REQ-12-002	SPS 12
TC_SPS_FP_001	4.7.1	devZ	SPS-REQ-10-001	SPS 10.2
TC_SPS_FP_002	4.7.2	devA	SPS-REQ-10-001	SPS 10.2
TC_SPS_PARA_001	5.1	devA	SPS-REQ-4-015	SPS 4.1
TC_SPS_PARA_002	5.2	devZ	SPS-REQ-4-014	SPS 4.1
TC_SPS_PARA_003	5.3	devZ	SPS-REQ-5-012	SPS 5.4
			SPS-REQ-6-011	SPS 6.2
			SPS-REQ-6-012	SPS 6.2
TC_SPS_PARA_004	5.4	devA	SPS-REQ-5-007	SPS 5.3
TC_SPS_PARA_005	5.5	devZ	SPS-REQ-4-013	SPS 4.4

Table 3: Test case to requirement mapping

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4 Test cases

4.1 Service Discovery (mDNS/DNS-SD)

4.1.1 TC_SPS_DISC_001: shippairing request announcement 1

Test Case ID: TC_SPS_DISC_001

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ announces a shippairing request upon configuration while devA is available (discoverable via SHIP service).

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to reject or ignore incoming TLS handshakes on the SHIP port*[1].
2. Configure the test tool not to initiate any SHIP connection to trusted SHIP nodes*[1].
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the DUT for SHIP and shippairing with the test tool.
5. Monitor the shippairing services for at most 5 minutes.

Expected result: During the last test step, the test tool receives and accepts the expected shippairing request of the DUT.

*[1] Note: Some conditions can lead to a situation where a shippairing request is not created or not recognized because the intended SHIP communication is established "too quickly". To ensure the detection of a shippairing request, a test case might require the test tool (as devA) to prevent the establishment of a SHIP communication at least for a while.

4.1.2 TC_SPS_DISC_002: shippairing request announcement 2

Test Case ID: TC_SPS_DISC_002

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ announces a shippairing request upon configuration – even if devA is not available.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- PRE_ttUnknownDut

Test steps:

1. Configure the DUT for SHIP and shippairing with the test tool.
2. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the DUT announces the corresponding shippairing request.
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.

Expected result: Within 5 minutes after the last test step, the test tool receives and accepts the shippairing request. Afterwards, a SHIP communication with the DUT is established.

4.1.3 TC_SPS_DISC_003: shippairing request deletion time 1

Test Case ID: TC_SPS_DISC_003

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ deletes a shippairing request according to PAR_spsSvcDelTime choice 1.

Test applicability: This test case applies if the DUT has the role devZ and the vendor filled PAR_spsSvcDelTime choice 1.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Configure the DUT for SHIP and shippairing with the test tool.
3. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing service instance for devA from the DUT. Do not monitor further and proceed with the next test step as soon as the SHIP connection is established.
4. Disable SHIP and SHIP Pairing Service functionality of the test tool.
5. Monitor the DUT's shippairing request for at most (PAR_spsSvcDelTime_DUR + 2) minutes.

Expected result: The shippairing request has been deleted during the monitoring of the last test step.

4.1.4 TC_SPS_DISC_004: shippairing request deletion time 2

Test Case ID: TC_SPS_DISC_004

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test verifies that devZ deletes a shippairing request according to PAR_spsSvcDelTime choice 2.

Test applicability: This test case applies if the DUT has the role devZ and the vendor filled PAR_spsSvcDelTime choice 2.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Configure the DUT for SHIP and shippairing with the test tool.
3. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing service instance for devA from the DUT. Do not monitor further and proceed with the next test step as soon as the SHIP connection is established.
4. Wait until the SHIP communication between DUT and test tool lasts for (PAR_spsSvcDelTime_DUR + 2) minutes without interruption.

Expected result: The shippairing request has been deleted.

4.1.5 TC_SPS_DISC_005: shippairing request deletion time 3

Test Case ID: TC_SPS_DISC_005

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ deletes a shippairing request not earlier than specified in PAR_spsSvcDelTime choice 2.

Test applicability: This test case applies if the DUT has the role devZ and the vendor filled PAR_spsSvcDelTime choice 2 AND if (PAR_spsSvcDelTime_DUR - 2) is not negative.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Configure the DUT for SHIP and shippairing with the test tool.
3. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing service instance for devA from the DUT. Do not monitor further and proceed with the next test step as soon as the SHIP connection is established.
4. Disable the SHIP and SHIP Pairing Service functionality of the test tool.
5. Monitor the shippairing request for (PAR_spsSvcDelTime_DUR + 2) minutes.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Expected result: The shippairing request is still announced by the DUT.

6. Enable the SHIP functionality of the test tool and wait until a SHIP communication with the DUT is established.
7. Wait until the SHIP communication lasts for (PAR_spsSvcDelTime_DUR - 2) minutes without interruption.
   Expected result: The shippairing request is still announced by the DUT.
8. Wait until the SHIP communication lasts for another 4 minutes without interruption.

Expected result: The shippairing request has been deleted.

4.1.6 TC_SPS_DISC_006: Immediate deletion of service upon revocation of trust

Test Case ID: TC_SPS_DISC_006

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ stops announcing the shippairing service immediately if the tester revokes trust for devA while the announcement is still active.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to reject or ignore incoming TLS handshakes on the SHIP port.
2. Configure the test tool not to initiate any SHIP connection to trusted SHIP nodes.
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the DUT for SHIP and shippairing with the test tool.
5. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing service instance for devA from the DUT.
6. Configure the DUT to revoke the trust for devA.
7. Monitor the shippairing services for at most 5 minutes.

Expected result: During the last monitoring phase, the DUT deletes the shippairing service instance for devA (by sending an mDNS goodbye packet with TTL=0). The service is not announced anymore.

4.1.7 TC_SPS_DISC_007: Verification of trust revocation

Test Case ID: TC_SPS_DISC_007

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test verifies that the deleted shippairing announcement in TC_SPS_DISC_006 indeed means that devZ does not trust devA anymore.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- Preconditions of TC_SPS_DISC_006 apply.
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint) beforehand.

Test steps:

1. Execute the test steps of TC_SPS_DISC_006.
   Expected result: The expected result of TC_SPS_DISC_006 must have been achieved.
2. Configure the test tool's SHIP functionality for regular SHIP behaviour*[1].
3. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s*[2].
4. Enable the SHIP functionality of the test tool (not the SHIP Pairing Service functionality).
5. Configure the test tool to trust the DUT (SHIP trust).
6. Monitor the test tool's SHIP connection state for 5 minutes.

*[1] Note: This means that the test tool does not apply anymore irregular configurations (from the test steps of TC_SPS_DISC_006) to ignore incoming TLS handshakes on the SHIP port or not to initiate SHIP connections to trusted SHIP nodes.**Expected result:** During the last monitoring phase, no SHIP connection is established between DUT and test tool*[2].

*[2] Note: The verdict of this test case does not soleily depend on the connection attempts of the test tool as TLS client (this would require that PAR_shipSvcByDevZ is set to true which might not be the case for a given DUT). It is also acceptable if the test tool does not recognize SHIP connections attempts of the DUT during the last monitoring step.

4.1.8 TC_SPS_DISC_008: Validation of TTL delay for reused instance name

Test Case ID: TC_SPS_DISC_008

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 5.5

Description: This test verifies that when devZ is configured to correct a faulty request by reusing the previous service instance name, it correctly enforces the minimum waiting period (1.5 times the original TTL) before re-announcing. This mechanism ensures consistency on the network side (devA caches).

Test applicability: This test case applies if the DUT has the role devZ and the vendor filled PAR_spsSvcCorrection with "Option 2".

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

1. Configure the test tool to log the service instance name and time and TTL of new shippairing requests for the test tool and goodbye messages (deletion) of a shippairing service instance for the test tool.
2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the DUT for SHIP and shippairing with the test tool using a wrong value for the devA-secret (not the correct devA-secret of the test tool).
4. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and rejects the shippairing requests for devA from the DUT.
5. Configure the DUT for SHIP and shippairing with the test tool using the correct devA-secret of the test tool.
6. Monitor the shippairing services for at most 5 minutes*[1].
   Expected result: During monitoring, the test tool receives and accepts the shippairing requests for devA from the DUT. Do not monitor further and proceed with the next test step as soon as the shippairing request was accepted.
7. Stop logging.
8. Evaluate the log:
   a. The time of the accepted shippairing request is called t_3.
   b. There is a goodbye message in the log with the same service instance name as used for the accepted shippairing request. The time of that goodbye message is called t_2.
   c. There is at least one rejected shipparing request in the log with the same service instance name as used for the accepted shippairing request. Choose the latest entry with this service instance name that has a time before t_2. The time of that latest rejected shippairing request is called t_1. The TTL of this shippairing request is called TTL_1.

*[1]: The maximum of 5 minutes holds true if the DUT's TTL does not exceed 3 minutes. If the vendor claims using longer TTLs or the test tool encounters a longer TTL in test step 4 (TTL_1), the maximum waiting time is extended to $2.5*TTL_1$.

Expected result: $t_2 + 1.5*TTL_1 \leq t_3$.

4.1.9 TC_SPS_DISC_009: Correction of wrong shippairing requests

Test Case ID: TC_SPS_DISC_009

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 5.5

Description: This test verifies that devZ can correct a faulty shippairing request for a given devA multiple times*[1].

*[1] Note:

- The test simulates that the administrator of devZ receives wrong credentials of devA and that these errors are corrected one after the other, each with an own shippairing request for devA. Only the last shippairing request will contain the correct credentials.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- The test does not require a particular value of PAR_spsSvcCorrection.
- The test also serves to create shippairing requests that can be analysed by test case TC_SPS_PARA_005.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Execute the following block of sub-steps four times.
   2.1. Configure the DUT for SHIP and shippairing with the test tool using a wrong value for the devA-secret (not the correct devA-secret of the test tool). With each block iteration, use a different (wrong) value for the devA-secret.
   2.2. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and rejects the shippairing requests for devA from the DUT.
3. Configure the DUT for SHIP and shippairing with the test tool using the correct devA-secret of the test tool.
4. Monitor the shippairing services for at most 5 minutes.

Expected result: During the monitoring of the last test step, the test tool receives and accepts the shippairing requests for devA from the DUT.

4.2 TXT record validation

4.2.1 TC_SPS_TXT_001: Announcement of valid TXT record

Test Case ID: TC_SPS_TXT_001

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 5.4, Table 1

Description: This test verifies that devZ creates a complete and syntactically correct TXT record.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Configure the DUT for SHIP and shippairing with the test tool.
3. Monitor the shippairing services for at most 5 minutes.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Expected result: During monitoring, the test tool receives the shippairing service instance for devA from the DUT. Do not monitor further and proceed with the next test step as soon as the SHIP connection is established.

4. Evaluate the TXT record of the DUT:
   a. Verify that the key txtvers is in first place.
   b. Verify that all mandatory keys (txtvers, parType, forId, forPar, trustId, trustPar, trustCurve, type, trustNonce, alg, digest) are present and occur not more than one time. The key name comparison must be done case insensitive*[1].
   c. Verify that the value of each key conforms to the format specified in [SHIPPairingService] Table 1 (e.g., txtvers is "1", parType is "fpSha256", fingerprints are 64 hex characters, etc.).
   d. Verify that the content of the TXT record matches the configured credentials of devA as well as the SHIP ID and fingerprint of devZ.
   e. Verify that the digest is calculated correctly.

Expected result: The TXT record contains all mandatory keys with correctly formatted values in the correct order and matching content.

*[1] Note: [RFC6763] specifies that key name comparisons are case-insensitive.

4.2.2 TC_SPS_TXT_002: Acceptance of rearranged TXT record

Test Case ID: TC_SPS_TXT_002

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.4

Description: This test verifies that devA accepts a TXT record where keys after txtvers are arranged in a different order than shown in [SHIPPairingService]*[1].

*[1] Note: This refers to Table 1 of [SHIPPairingService]. "txtvers" must be in first place.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool (devZ) to announce shippairing requests where the TXT record contains the keys in the following order: "txtvers", "parType", "trustId", "forPar", "forId", "trustPar", "type", "trustCurve", "alg", "digest", "trustNonce".
2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the test tool for SHIP and shippairing with the DUT.
4. Monitor the test tool's SHIP connection state for at most 5 minutes.

Expected result: During the last monitoring phase, a SHIP connection is established between the DUT and the test tool.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.2.3 TC_SPS_TXT_003: Rejection of incomplete TXT record

Test Case ID: TC_SPS_TXT_003

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.4

Description: This test verifies that devA correctly rejects a TXT record that is missing a mandatory key.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s. This configuration applies whenever the SHIP functionality of the test tool is (re-)enabled.
2. Execute the following block of sub-steps for each key of the list [txtvers, parType, forId, forPar, trustId, trustPar, trustCurve, type, trustNonce, alg, digest]. The number of the block iteration is counted with the variable i and starts with the first key (i=1). The key of the current iteration i is called key_i.
   2.1. Configure the test tool (devZ) to announce shippairing requests where the TXT record does NOT contain key_i. Despite the absence of key_i in the TXT record, the test tool uses all keys that are required to correctly calculate the value of the key "digest" according to [SHIPPairingService].
   2.2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
   2.3. Configure the test tool for SHIP and shippairing with the DUT.
   2.4. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
   Expected result: The DUT rejects each SHIP connection attempt of the test tool.
   2.5. Configure the test tool to stop sending the shippairing request and to send a "goodbye message" for the service instance instead.

Expected result: All executed test steps completed successfully.

4.2.4 TC_SPS_TXT_004: Rejection of malformed TXT record 1

Test Case ID: TC_SPS_TXT_004

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.4

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test verifies that devA correctly rejects a TXT record that is malformed (extended/curtailed key names, wrong value formats). The test assumes that the digest was calculated with correctly formatted input.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
2. Configure the test tool not to reuse shippairing service instance names when sending a different TXT record than before*[1].
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the test tool for SHIP (not shippairing) with the DUT.
5. Prepare key-value pairs for shippairing with the DUT. At this stage, all key-value pairs are correct*[2]. The set of these key-value pairs is called TXT_Z_a.
6. Execute the following block of sub-steps for each key of Table 4. The number of the block iteration is counted with the variable i. The key of the current iteration i is called key_i.
   6.1. Execute the following block of sub-steps for each applicable error of key_i of Table 4. The block iteration is counted with the variable k. The applicable error of the current iteration is called error_k.
      6.1.1. Create TXT_Z_b as copy of TXT_Z_a.
      6.1.2. In TXT_Z_b, modify the key-value pair of key_i according to error_k as described in Table 1 (see section 2.8.9).
      6.1.3. Configure the test tool to announce a shippairing request using TXT_Z_b.
      6.1.4. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
      Expected result: The DUT rejects each SHIP connection attempt of the test tool.
      6.1.5. Configure the test tool to stop sending the shippairing request and to send a "goodbye message" for the service instance instead.

*[1] Note: This corresponds to "option 1" in [SHIPPairingService] section 5.5.

*[2] Note: This means that the (correct) digest is calculated based upon correctly written key names and correct values, just as specified in [SHIPPairingService]. This test does not calculate a digest based upon wrongly written key names or incorrect values.

Expected result: All executed test steps completed successfully.

key	Applicable errors
txtvers	E01, E02, E03, E12
parType	E01, E02, E04, E12
forId	E01, E02, E13
forPar	E01, E02, E05, E07, E08
trustId	E01, E02, E13
trustPar	E01, E02, E05, E07, E08
trustCurve	E01, E02, E09, E12

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

type	E01, E02, E10, E12
trustNonce	E01, E02, E06, E07, E08
alg	E01, E02, E11, E12
digest	E01, E02, E05, E07, E08

Table 4: Applicable errors per key

4.2.5 TC_SPS_TXT_005: Ignore unknown keys

Test Case ID: TC_SPS_TXT_005

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapter 5.4

Description: This test verifies that devA is robust and ignores any additional, unexpected key-value pairs (which are not defined in [SHIPPairingService] Table 1), thereby ensuring forward compatibility with future protocol extensions.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool (devZ) to announce shippairing requests where the TXT record contains all mandatory keys, plus two extra, undefined key-value pairs (e.g., futurekey1=123 and futurekey2=456). The first extra key-value pair is located after txtvers, the second at the end of the regular key-value pairs.
2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the test tool for SHIP and shippairing with the DUT.
4. Monitor the test tool's SHIP connection state for at most 5 minutes.

Expected result: The DUT successfully establishes a SHIP connection with the test tool.

4.2.6 TC_SPS_TXT_006: Reject unsupported elliptic curve (trustCurve)

Test Case ID: TC_SPS_TXT_006

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.4

Description: This test verifies that devA rejects a pairing request from a devZ that proposes to use an elliptic curve not supported by devA.

Test applicability: This test case applies if the DUT has the role devA and the vendor set in PAR_eccOfDevA at least for one elliptic curve the option "not supported".

Preconditions:

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- Ensure that no other IP device than the DUT will access the SHIP port of the test tool*[1].

*[1] Note: This test case makes use of unsupported credentials which makes it more difficult to distinguish connection attempts from a DUT from those of other SHIP nodes. Having only test tool and DUT in an isolated network segment is the simplest possibility to achieve the precondition.

Test steps:

1. Configure the test tool not to initiate any SHIP connection to trusted SHIP nodes.
2. For the creation of a shippairing request, take the DUT's credentials and apply the following adjustments for the configuration of the test tool:
   a. If PAR_eccOfDevA has brainpoolP256r1 set to "not supported": Set the TXT record key "forPar" to the DUT's fingerprint for secp256r1. Set the TXT record key "trustCurve" to "brainpoolP256r1".
   b. If PAR_eccOfDevA has brainpoolP256r1 set to "supported" and brainpoolP384r1 set to "not supported": Set the TXT record key "forPar" to the DUT's fingerprint for brainpoolP256r1. Set the TXT record key "trustCurve" to "brainpoolP384r1".
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the test tool for SHIP and shippairing with the DUT based upon the adjustments described above*[2].
5. Monitor the the test tool's SHIP server port for 5 minutes.

*[2] Note: This also means that the calculation of the digest considers the adjustments of the DUT's credentials.

Expected result: During the monitoring of the last test step, the test tool did not receive any connection attempt on its SHIP server port*[3].

*[3] Note: In a test environment it can be assumed that only the DUT tries to connect with the SHIP server port of the test tool and that this includes a TLS "Client Hello" message.

4.2.7 TC_SPS_TXT_007: Prevent spoofing from TXT record with duplicate trustPar

Test Case ID: TC_SPS_TXT_007

Roles: DUT = devA. Test tool = devZ.

Reference: Security aspect of [SHIPPairingService]*[1]

Description: This test primarily verifies that devA is not susceptible to an attack vector where correct data is combined with alternative credentials (esp. an adversary's certificate fingerprint). In general, this means that devA must either discard the alternative credentials (i.e. use only the correct data) or it must consider the whole TXT record being invalid. But devA must not accept the correct data and additionally accept the alternative credentials*[1]. The test focuses on the rejection of an adversary's certificate fingerprint.

*[1] Note: The case of multiple occurrences of a key like "trustPar" is not explicitly mentioned in [SHIPPairingService]. TXT records with a duplicate key name and different values are ambigious and

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

might trigger an unprepared implementation to consider data valid and trust an adversary's credential. This test case does not interfere with rules of [SHIPPairingService] and just adds another security aspect that must be fulfilled by a devA to pass this test specification.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint, devA-secret) beforehand.

Test steps:

1. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
2. Execute the following block of sub-steps two times. The number of the block iteration is counted with the variable i and starts with i=1:
   - 2.1. For a (virtual) SHIP instance devZ1: Create new credentials CRE_Z_1. This includes a new certificate (i.e. a certificate that has not been used before) and a new SHIP ID.
   - 2.2. For a (subsequent) SHIP instance devZ2: Create new credentials CRE_Z_2. This includes a new certificate (i.e. a certificate that has not been used before). The SHIP ID shall be the same as in CRE_Z_1.
   - 2.3. Create a random trustNonce value.
   - 2.4. Create a sequence (i.e. ordered list) of key-value pairs TXT_Z_1 as basis for a (virtual) shippairing request: All key-value pairs shall be set as if devZ1 would send a regular shippairing request to the DUT. This means that TXT_Z_1 contains only unique key names. The digest is calculated using the previously created trustNonce value.
     Note: TXT_Z_1 just serves to create TXT_Z_2. TXT_Z_1 will not be used directly for a shippairing request.
   - 2.5. Create a sequence TXT_Z_2 as copy of TXT_Z_1. Then, modify TXT_Z_2: Insert another "trustPar" key with the certificate fingerprint of CRE_Z_2 at the following position: Insert it right before the already existing "trustPar" entry if the current block iteration i is 1. Insert it at a random position after the already existing "trustPar" entry if the current block iteration i is 2.
   - 2.6. Configure the test tool to have a SHIP instance devZ2 with the credentials CRE_Z_2. Enable the SHIP and SHIP Pairing Service functionality of devZ2 only and disable other SHIP instances of the test tool.
   - 2.7. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_2.
   - 2.8. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
     Expected result: The DUT rejects each SHIP connection attempt of the test tool.
   - 2.9. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.
   - 2.10. If the current block iteration i is 1: Wait for 18 minutes (see section 2.8.8 to shorten this period).

Expected result: All executed test steps completed successfully.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.2.8 TC_SPS_TXT_008: Prevention of secret leakage in TXT record

Test Case ID: TC_SPS_TXT_008

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 5.4

Description: This test verifies that devZ does not expose the devA-secret (or the trustNonce using the secret's value) in the shippairing TXT record.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
2. Configure the DUT for SHIP and shippairing with the test tool.
3. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing service instance for devA from the DUT.
4. Convert the known devA-secret into its string representation.
5. Scan every key and every value within the captured TXT record for the occurrence of the devA-secret string.

Expected result: The devA-secret string is not found in any key or value of the TXT record (case-insensitive sub-string comparison).

4.2.9 TC_SPS_TXT_009: Rejection of malformed TXT record 2

Test Case ID: TC_SPS_TXT_009

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.4

Description: This test verifies that devA correctly rejects a TXT record that is malformed (wrong value formats). The test assumes that the digest was calculated with already malformed input.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
2. Configure the test tool not to reuse shippairing service instance names when sending a different TXT record than before*[1].

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the test tool for SHIP (not shippairing) with the DUT.
5. Prepare key-value pairs for shippairing with the DUT, except for the key "digest". At this stage, all key-value pairs are correct*[2]. The set of these key-value pairs is called TXT_Z_a.
6. Execute the following block of sub-steps for each key of Table 5. The number of the block iteration is counted with the variable i. The key of the current iteration i is called key_i.
   6.1. Execute the following block of sub-steps for each applicable error of key_i of Table 5. The block iteration is counted with the variable k. The applicable error of the current iteration is called error_k.
      6.1.1. Create TXT_Z_b as copy of TXT_Z_a.
      6.1.2. In TXT_Z_b, modify the key-value pair of key_i according to error_k as described in Table 1 (see section 2.8.9).
      6.1.3. Calculate the digest using the values of TXT_Z_b and append the key-value pair of the digest to TXT_Z_b.
      6.1.4. Configure the test tool to announce a shippairing request using TXT_Z_b.
      6.1.5. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
      Expected result: The DUT rejects each SHIP connection attempt of the test tool.
      6.1.6. Configure the test tool to stop sending the shippairing request and to send a "goodbye message" for the service instance instead.

*[1] Note: This corresponds to "option 1" in [SHIPPairingService] section 5.5.

*[2] Note: In this test case the digest is calculated in a later test step based upon incorrect values.

Expected result: All executed test steps completed successfully.

key	Applicable errors
txtvers	E03, E12
parType	E04, E12
forId	E13
forPar	E05, E07, E08
trustId	E12
trustPar	E05, E07, E08
trustCurve	E09, E12
type	E10, E12
trustNonce	E08, E14, E15
alg	E11, E12

Table 5: Applicable errors per key

4.3 Digest calculation and verification

4.3.1 TC_SPS_DIGEST_001: Digest verification

Test Case ID: TC_SPS_DIGEST_001

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapters 7, 9

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test verifies if devA correctly verifies the digest of a shippairing request.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint, devA-secret) beforehand.

Overview on test steps and test phases:
The test case is split into several phases. Phase 1 creates configurations for different SHIP instances of the test tool and proper shippairing requests. When a subsequent phase announces a shippairing request, the required key-value pairs are taken from pre-configured data of phase 1. This also includes "trustNonce" and "digest". Phase 1 creates 5 shippairing requests with a valid digest and 5 shippairing requests with an invalid digest. Phase 2 verifies that shippairing requests with valid digest are accepted, while the other shippairing requests are not accepted.

Test steps:

1. Phase 1 – preparation for test tool instances:
   1.1. Prepare configurations for 10 distinct SHIP instances devZ_m of the test tool, with $1 \leq m \leq 10$. Prepare also shippairing requests for each devZ_m. The shippairing requests must have distinct trustNonce values. 5 of the shippairing requests shall have a correctly calculated digest. The other 5 shippairing requests shall have digests with invalid values (but correctly formatted). Which of the 10 shippairing requests contain valid or invalid digest values shall be chosen at random and is then noted for a later evaluation.
   1.2. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s. This configuration applies whenever the SHIP functionality of the test tool is (re-)enabled.
2. Phase 2 – pairing attempts: Execute the following block of sub-steps 10 times. The number of the block iteration is counted with the variable i and starts with i=1:
   2.1. Configure the test tool to have a SHIP instance devZ_i. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
   2.2. Configure devZ_i for SHIP and shippairing with the DUT, using the shippairing request i of phase 1.
   2.3. Monitor the SHIP connection state of devZ_i for 5 minutes (see section 2.8.7 to shorten this period).
   Expected result: The SHIP communication between devZ_i and DUT is established if the configuration i contains a valid digest. The SHIP communication between devZ_i and DUT is not established if the configuration i contains an invalid digest.
   2.4. Stop the SHIP communication and disable the SHIP and shippairing functionality of devZ_i.
   2.5. If i is smaller than 10, ensure the DUT can accept another shippairing request. This can be achieved by waiting for 18 minutes or by a manual configuration on the DUT to remove the trust in devZ_i.

Expected result: All executed test steps completed successfully.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.3.2 TC_SPS_DIGEST_002: Ring buffer properties and blocking of replay attempts

Test Case ID: TC_SPS_DIGEST_002

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapter 11

Description: This test verifies that devA fulfills the ring buffer characteristics specified in PAR_ringBufferSize (ring buffer size) and its usage (block replay attempts of old requests; circular (over-)writing characteristic of the buffer).

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint, devA-secret) beforehand.

Overview on test steps and test phases:
The test case is split into several phases. Phase 1 creates configurations for different SHIP instances of the test tool and proper shippairing requests. When a subsequent phase announces a shippairing request, the required key-value pairs are taken from pre-configured data of phase 1. This also includes "trustNonce" and "digest".

Test steps:

1. Phase 1 – preparation for test tool SHIP instances:
   1.1. Prepare configurations for (PAR_ringBufferSize +1) distinct SHIP instances of the test tool: For each m, with 1 ≤ m ≤ PAR_ringBufferSize+1, create the credentials CRE_Z_m and the key-value-set TXT_Z_m:
      1.1.1. The credentials CRE_Z_m contain the SHIP ID and (per supported curve) certificate, SKI, and fingerprint. Each SHIP ID and each certificate must be unique.
      1.1.2. The key-value-set TXT_Z_m contains all key-value pairs that will be used to create a shippairing request to the DUT. The keys "trustId" and "trustPar" are set according to CRE_Z_m. The key "trustNonce" is set with a random value. Each "trustNonce" value must be unique. The other keys are set as required by [SHIPPairingService] and the credentials of the DUT.
   1.2. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 5s. This configuration applies whenever the SHIP functionality of the test tool is (re-)enabled.
2. Phase 2 – filling of the ring buffer and verify blocking of random replay attempts: Execute the following block of sub-steps PAR_ringBufferSize times. The number of the block iteration is counted with the variable i and starts with i=1:
   2.1. Configure the test tool to have a SHIP instance with own credentials CRE_Z_i. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
   2.2. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_i.
   2.3. Monitor the test tool's SHIP connection state for at most 5 minutes.
      Expected result: The SHIP communication between test tool and DUT is established.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

2.4. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.
2.5. Wait for 18 minutes (see section 2.8.8 to shorten this period).
2.6. If the current block iteration i is 2 or greater, execute the following sub-steps:
2.6.1. Use a random number generator to get a random integer k in the range [1, i-1].
2.6.2. Configure the test tool to have a SHIP instance with own credentials CRE_Z_k. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
2.6.3. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_k.
2.6.4. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
Expected result: The DUT rejects each SHIP connection attempt of the test tool.
2.6.5. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.

3. Phase 3 – verify the circular (over-)writing characteristic of the buffer: Execute the following sub-steps:
3.1. Configure the test tool to have a SHIP instance with own credentials CRE_Z_(PAR_ringBufferSize+1).
3.2. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_(PAR_ringBufferSize+1). Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
3.3. Monitor the test tool's SHIP connection state for at most 5 minutes.
Expected result: The SHIP communication between test tool and DUT is established.
3.4. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.
3.5. Wait for 18 minutes (see section 2.8.8 to shorten this period).
3.6. Configure the test tool to have a SHIP instance with own credentials CRE_Z_1.
3.7. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_1. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
3.8. Monitor the test tool's SHIP connection state for at most 5 minutes.
Expected result: The SHIP communication between test tool and DUT is established.

Expected result: All executed test steps completed successfully.

4.3.3 TC_SPS_DIGEST_003: Ring buffer persistence

Test Case ID: TC_SPS_DIGEST_003

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapter 11

Description: This test verifies that devA stores its ring buffer persistently.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint, devA-secret) beforehand.

Overview on test steps and test phases:
The test case is split into several phases. Phase 1 creates configurations for different SHIP instances of the test tool and proper shippairing requests. When a subsequent phase announces a shippairing request, the required key-value pairs are taken from pre-configured data of phase 1. This also includes "trustNonce" and "digest".

Test steps:
1. Phase 1 – preparation for test tool SHIP instances:
   1.1. Prepare configurations for 3 distinct SHIP instances of the test tool: For each m, with 1 ≤ m ≤ 3, create the credentials CRE_Z_m and the key-value-set TXT_Z_m:
      1.1.1. The credentials CRE_Z_m contain the SHIP ID and (per supported curve) certificate, SKI, and fingerprint. Each SHIP ID and each certificate must be unique.
      1.1.2. The key-value-set TXT_Z_m contains all key-value pairs that will be used to create a shippairing request to the DUT. The keys "trustId" and "trustPar" are set according to CRE_Z_m. The key "trustNonce" is set with a random value. Each "trustNonce" value must be unique. The other keys are set as required by [SHIPPairingService] and the credentials of the DUT.
   1.2. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 5s. This configuration applies whenever the SHIP functionality of the test tool is (re-)enabled.
2. Phase 2 – filling of the ring buffer: Execute the following block of sub-steps 3 times. The number of the block iteration is counted with the variable i and starts with i=1:
   2.1. Configure the test tool to have a SHIP instance with own credentials CRE_Z_i. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
   2.2. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_i.
   2.3. Monitor the test tool's SHIP connection state for at most 5 minutes.
      Expected result: The SHIP communication between test tool and DUT is established.
   2.4. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.
   2.5. If i is less than 3: Wait for 18 minutes (see section 2.8.8 to shorten this period).
3. Phase 3 – verify persistence of the ring buffer and blocking of the two eldest replay attempts: Execute the following sub-steps:
   3.1. Power-cycle the DUT.
   3.2. Configure the test tool to have a SHIP instance with own credentials CRE_Z_1. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
   3.3. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_1.
   3.4. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
      Expected result: The DUT rejects each SHIP connection attempt of the test tool.
   3.5. Stop the SHIP communication and disable the SHIP and shippairing functionality of the test tool.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3.6. Configure the test tool to have a SHIP instance with own credentials CRE_Z_2. Enable SHIP and SHIP Pairing Service functionality of the test tool for this SHIP instance only and disable other SHIP instances of the test tool.
3.7. Configure the test tool for SHIP and shippairing with the DUT using TXT_Z_2.
3.8. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
3.9. **Expected result:** The DUT rejects each SHIP connection attempt of the test tool.

Expected result: All executed test steps completed successfully.

4.4 Process flow and state management

4.4.1 TC_SPS_PROC_001: Single control unit rule 1

Test Case ID: TC_SPS_PROC_001

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 4.3, chapter 9

Description: This test verifies that devA does not accept a second addCu request while it is already paired with a control unit.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- Preconditions of TC_SPS_TRUST_001 apply. The credentials (SHIP ID, certificate(s)) of the test tool itself used for TC_SPS_TRUST_001 are called CRE_Z_1 and represent a devZ1.

Test steps:

1. Execute the test steps of TC_SPS_TRUST_001 with test run i=2.
   Expected result: The expected result of TC_SPS_TRUST_001 must have been achieved.
2. Configure the test tool with an additional SHIP instance devZ2 with own credentials CRE_Z_2 (different to CRE_Z_1).
3. Configure the test tool's SHIP instance devZ2 for SHIP and shippairing with the DUT.
4. Ensure that the test tool's SHIP instance devZ2 tries to establish a SHIP connection with the DUT repeatedly and that it announces a shippairing request.
5. Ensure that the first SHIP instance of the test tool (devZ1) keeps the SHIP communication with the DUT established or tries to re-establish the SHIP communication with the DUT in case of interrupts.
6. Monitor the test tool's SHIP connection states of devZ1 and devZ2 for 5 minutes.

Expected result: The DUT has no SHIP communication established with devZ2. The SHIP communication between devA and devZ1 is established.

4.4.2 TC_SPS_PROC_002: Single control unit rule 2

Test Case ID: TC_SPS_PROC_002

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 4.3

Description: This test verifies that devA does not accept a second addCu request too early.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- Preconditions of TC_SPS_TRUST_001 apply. The credentials (SHIP ID, certificate(s)) of the test tool itself used for TC_SPS_TRUST_001 are called CRE_Z_1 and represent a devZ1.

Test steps:

1. Execute the test steps of TC_SPS_TRUST_001 with test run i=2.
   Expected result: The expected result of TC_SPS_TRUST_001 must have been achieved.
2. Configure the test tool with an additional SHIP instance devZ2 with own credentials CRE_Z_2 (different to CRE_Z_1).
3. Configure the test tool's SHIP instance devZ2 for SHIP and shippairing with the DUT.
4. Ensure that the test tool's SHIP instance devZ2 tries to establish a SHIP connection with the DUT repeatedly and that it announces a shippairing request.
5. Disable the test tool's SHIP instance devZ1.
6. Monitor the test tool's SHIP connection state of devZ2 for 13 minutes.

Expected result: The DUT has no SHIP communication established with devZ2.

4.4.3 TC_SPS_PROC_003: addCu-request process (listener) reactivation after communication loss

Test Case ID: TC_SPS_PROC_003

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 4.3

Description: This test verifies that devA accepts a further shippairing request if it has a "currently trusted devZ", but no SHIP communication with this devZ for 15 minutes.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- Preconditions of TC_SPS_TRUST_001 apply. The credentials (SHIP ID, certificate(s)) of the test tool itself used for TC_SPS_TRUST_001 are called CRE_Z_1 and represent a devZ1.

Test steps:

1. Execute the test steps of TC_SPS_TRUST_001 with test test run i=2.
   Expected result: The expected result of TC_SPS_TRUST_001 must have been achieved.
2. Disable SHIP and SHIP Pairing Service functionality of the test tool's SHIP instance devZ1.
3. Configure the test tool with a new SHIP instance devZ2 with own credentials CRE_Z_2 (different to CRE_Z_1).
4. Enable SHIP and SHIP Pairing Service functionality of devZ2.
5. Configure the test tool's SHIP instance devZ2 for SHIP and shippairing with the DUT.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

6. Ensure that the test tool's SHIP instance devZ2 tries to establish a SHIP connection with the DUT repeatedly and that it announces a shippairing request.
7. Wait for 18 minutes since the end of the previous SHIP communication (devZ1, using CRE_Z_1) with the DUT.
8. Monitor the test tool's SHIP connection state of devZ2 (using CRE_Z_2) with the DUT for at most 5 minutes.
   Expected result: During the monitoring, the DUT has a SHIP communication with the test tool's SHIP instance devZ2.

Expected result: All test steps completed successfully.

4.4.4 TC_SPS_PROC_004: Evaluation of corrected request

Test Case ID: TC_SPS_PROC_004

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 5.5

Description: This test verifies that devA processes a new, corrected request after a previously received invalid request.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the test tool for SHIP and shippairing with the DUT using an invalid devA-secret (i.e. NOT the devA-secret of the DUT).
4. Monitor the test tool's SHIP connection state for 5 minutes (see section 2.8.7 to shorten this period).
   Expected result: The DUT rejects each SHIP connection attempt of the test tool.
5. Configure the test tool to stop sending the shippairing request and to send a "goodbye message" for the first service instance instead.
6. Configure the test tool to immediately send a second, completely valid shippairing request (i.e. with the correct devA-secret of the DUT). This new shippairing request shall use a different service instance name than the previous shippairing request.
7. Monitor the test tool's SHIP connection state for at most 5 minutes.

Expected result: The SHIP communication between test tool and DUT is established.

4.4.5 TC_SPS_PROC_005: Sending of parallel shippairing requests

Test Case ID: TC_SPS_PROC_005

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.4

Description: This test verifies that devZ uses unique service instance names when sending multiple, parallel pairing requests to different devA devices.

Test applicability: This test case applies if the DUT has the role devZ and the vendor filled PAR_parallelSpsSvcByDevZ with "yes".

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to have two own SHIP instances devA1 and devA2.
2. Configure the test tool to reject or ignore incoming TLS handshakes on the SHIP ports of devA1 and devA2.
3. Configure the DUT (devZ) for SHIP and shippairing with both devA1 and devA2 simultaneously.
4. Monitor the shippairing services for at most 5 minutes.
   Expected result: The DUT did broadcast two distinct shippairing service instances with the following properties:
   a. The service instance names are unique (e.g., by appended counters "#1" and "#2" as recommended in the specification).
   b. The TXT record of one instance has forId set to the SHIP ID of devA1.
   c. The TXT record of another instance has forId set to the SHIP ID of devA2.

Expected result: The verification of the last test step is successful.

4.4.6 TC_SPS_PROC_006: Sequential processing of parallel shippairing requests

Test Case ID: TC_SPS_PROC_006

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapter 9

Description: This test verifies that devA accepts not more than one of several shippairing requests received in parallel.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to repeat unsuccessful connection attempts latest within 5s.
2. Configure the test tool to have two own SHIP instances devZ1 and devZ2.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

3. Configure devZ1 and devZ2 for SHIP and shippairing with the DUT (this also means that both instances get the secret of the DUT).
4. Enable SHIP and SHIP Pairing Service functionality of the test tool for devZ1 and devZ2. The methodology should result in almost simultaneous shippairing requests of devZ1 and devZ2.
5. Monitor the SHIP connection state of both SHIP instances of the test tool for 5 minutes.

Expected result: Exactly one of the test tool's SHIP instances has a SHIP connection established with the DUT, while each connection attempt of the other SHIP instance is rejected.

4.4.7 TC_SPS_PROC_007: SHIP connection attempt before shippairing request

Test Case ID: TC_SPS_PROC_007

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ tries to establish actively (i.e. initiate) a SHIP connection before sending the shippairing request.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Configure the test tool to log incoming SHIP connection attempts. Each entry of the log needs to contain the time of the connection attempt and the certificate of the TLS client*[1].
2. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the DUT for SHIP and shippairing with the test tool.
4. Wait until the test tool receives and accepts a shippairing request of the DUT. The time of this event is called t_1.
5. Wait for 2 more seconds.
6. Stop logging incoming SHIP connection attempts at the test tool.

*[1] Note: The certificate is taken from the TLS message "Client Certificate", see PHR_002.

Expected result: The log of the incoming SHIP connections attempts contains an entry with the certificate of the DUT and a time that is not later than t_1+2s *[2].

*[2] Note: This test tolerates if a SHIP connection attempt of the DUT is recognised not before the shippairing request but at least not later than 2 seconds thereafter in the event that latencies cause the sequence of transmitted messages to be incorrect.

Test repetition: The test passes if at least 1 of up to 10 iterations of TC_SPS_PROC_007 yield the expected result.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.4.8 TC_SPS_PROC_008: SHIP connection acceptance before shippairing request

Test Case ID: TC_SPS_PROC_008

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devZ permits to establish passively (i.e. accept) a SHIP connection before sending the shippairing request.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut
- PRE_devZShipPort
- The tester must know the DUT credentials (SHIP ID, SKI/fingerprint) beforehand.

Test steps:

1. Configure the test tool to reject or ignore incoming TLS handshakes on the SHIP port.
2. Configure the test tool to repeat unsuccessful connection attempts latest within 1s.
3. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
4. Configure the test tool to trust the DUT (SHIP trust).
5. Configure the DUT for SHIP and shippairing with the test tool.
6. Wait until the following condition is fulfilled: The test tool receives and accepts a shippairing request of the DUT or a SHIP connection is established between test tool and DUT.
7. Execute this test step if and only if NO SHIP connection is established with the DUT yet: Wait for 2 more seconds*[1].

Expected result: A SHIP connection is established between the test tool and the DUT.

*[1] Note: This test tolerates if a SHIP connection attempt of the test tool is accepted by the DUT not before the shippairing request but at least not later than 2 seconds thereafter in the event that latencies cause the sequence of transmitted messages to be incorrect.

Test repetition: The test passes if at least 1 of up to 20 iterations of TC_SPS_PROC_008 yield the expected result.

4.4.9 TC_SPS_PROC_009: addCu-request process (listener) abortion upon re-established communication

Test Case ID: TC_SPS_PROC_009

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 4.3

Description: This test verifies that devA aborts a reactivated addCu-request process if the communication with the already trusted devZ1 is re-established (i.e. before a new valid shippairing request of a devZ2 could be processed).

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- Preconditions of TC_SPS_TRUST_001 apply. The credentials (SHIP ID, certificate(s)) of the test tool itself used for TC_SPS_TRUST_001 are called CRE_Z_1 and represent a devZ1.

Test steps:

1. Execute the test steps of TC_SPS_TRUST_001 with test run i=2.
   Expected result: The expected result of TC_SPS_TRUST_001 must have been achieved.
2. Stop the SHIP communication of devZ1 and disable the SHIP and shippairing functionality of devZ1.
3. Wait for 18 minutes (see section 2.8.8 to shorten this period).
4. Enable the SHIP functionality of devZ1 (not the shippairing functionality).
5. Monitor the test tool's SHIP connection state for at most 5 minutes.
   Expected result: The SHIP communication between test tool and DUT is established
6. Configure the test tool with an additional SHIP instance devZ2 with own credentials CRE_Z_2 (different to CRE_Z_1).
7. Configure the test tool to repeat unsuccessful SHIP connection attempts of devZ2 latest within 60s.
8. Enable SHIP and SHIP Pairing Service functionality of the test tool for devZ2.
9. Configure the test tool's SHIP instance devZ2 for SHIP and shippairing with the DUT.
10. Ensure that the test tool's SHIP instance devZ2 tries to establish a SHIP connection with the DUT repeatedly and that it announces a shippairing request.
11. Monitor the test tool's SHIP connection states of devZ1 and devZ2 for 5 minutes.

Expected result: The DUT has no SHIP communication established with devZ2. The SHIP communication between devA and devZ1 is established.

4.5 Trust update and management

4.5.1 TC_SPS_TRUST_001: devA SHIP communication after accepted shippairing request

Test Case ID: TC_SPS_TRUST_001

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 4.2

Description: This test verifies that devA has a SHIP connection with devZ upon successful evaluation of the shippairing request.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test execution: TC_SPS_TRUST_001 can be executed with test run i=1 or test run i=2. For a positive test verdict of TC_SPS_TRUST_001 itself, both test runs must be completed successfully. For each test run, the preconditions must be fulfilled again.

Test steps:

1. Configure the test tool:
   a. If test run i=1: Configure the test tool not to initiate any SHIP connection to trusted SHIP nodes. The test tool will accept a valid incoming SHIP connection of a trusted SHIP node.
   b. If test run i=2: The test tool can try to initiate SHIP connections to trusted SHIP nodes. The test tool will accept a valid incoming SHIP connection of a trusted SHIP node.
2. Enable SHIP and SHIP Pairing Service functionality of the test tool.
3. Configure the test tool for SHIP and shippairing with the DUT.
4. Ensure that the test tool sends a shippairing request.

Expected result: Within 5 minutes after the last test step, the DUT initiates (i=1 or 2) or accepts (i=2) a SHIP connection with the test tool.

4.5.2 TC_SPS_TRUST_002: Removal of trust for previous control unit

Test Case ID: TC_SPS_TRUST_002

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 10.3

Description: This test verifies that devA removes trust in a previously paired control unit (devZ1) after successfully pairing with a new one (devZ2).

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- Preconditions of TC_SPS_PROC_003 apply. The credentials (SHIP ID, certificate(s)) of the test tool's SHIP instances used for TC_SPS_PROC_003 are called CRE_Z_1 for devZ1 and CRE_Z_2 for devZ2.

Test steps:

1. Execute the test steps of TC_SPS_PROC_003.
   Expected result: The expected result of TC_SPS_PROC_003 must have been achieved.
2. Disable SHIP and SHIP Pairing Service functionality of the test tool's SHIP instance devZ2.
3. Configure the test tool to repeat unsuccessful connection attempts latest within 60s.
4. Enable the SHIP functionality of the test tool's SHIP instance devZ1. Ensure that devZ1 still trusts the DUT (no further shippairing request is sent) and tries to establish a SHIP connection with the DUT.
5. Monitor the SHIP connection state of devZ1 for 5 minutes (see section 2.8.7 to shorten this period).

Expected result: The connection attempts from devZ1 are rejected by the DUT.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

4.6 QR code and credential provisioning

4.6.1 TC_SPS_QR_001: Mandatory QR code keys for SHIP Pairing Service

Test Case ID: TC_SPS_QR_001

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], chapter 12; [SRIP], section 3.1

Description: This test verifies that a device (devA) supporting the SHIP Pairing Service provides a QR code containing the necessary extensions.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- None.

Test steps:

1. Obtain the QR code of the DUT.
2. Scan the QR code and parse its content.

Expected result: The following keys are part of the QR code*[1]:

- The "ID" key with the SHIP ID of the DUT.
- The "FPH256" key with a correctly formatted SHA-256 fingerprint.
- The "SPSEC" key with a correctly formatted 128-bit secret.
- If the QR code contains the "BSKI" key: The "BFPH256" key with a correctly formatted SHA-256 fingerprint.
- If the QR code contains the "B2SKI" key: The "B2FPH256" key with a correctly formatted SHA-256 fingerprint.

*[1] Note: Only keys required for SHIP Pairing Service are listed here.

4.7 Authentication

4.7.1 TC_SPS_FP_001: Rejection of invalid fingerprint of devA

Test Case ID: TC_SPS_FP_001

Roles: DUT = devZ. Test tool = devA.

Reference: [SHIPPairingService], section 10.2 (and 6.2)

Description: This test verifies that devZ verifies the fingerprint of the devA certificate received during the TLS handshake.

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test steps:

1. Configure the test tool to accept a shippairing request even if the value of the "forPar" key differs from the fingerprint of the certificate of the test tool.
2. Configure the test tool to accept a shippairing request regardless of the value of the key "digest".
3. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
4. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
5. Create a random 256bit value that differs from the fingerprint of the test tool. This random value is called fp_rand.
6. Configure the DUT for SHIP and shippairing with the test tool, but use fp_rand for "forPar" instead of the fingerprint of the test tool.
7. Monitor the shippairing services for at most 5 minutes.
   Expected result: During monitoring, the test tool receives and accepts the shippairing request for devA from the DUT.
8. Monitor the test tool's SHIP connection state for 5 minutes.

Expected result: During the last monitoring phase, no SHIP connection is established between DUT and test tool.

4.7.2 TC_SPS_FP_002: Rejection of invalid fingerprint of devZ

Test Case ID: TC_SPS_FP_002

Roles: DUT = devA. Test tool = devZ.

Reference: [SHIPPairingService], section 10.2 (and 6.2)

Description: This test verifies that devA verifies the fingerprint of the devZ certificate received during the TLS handshake.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- PRE_dutUnknownTt
- PRE_ttUnknownDut

Test steps:

1. Create a random 256bit value that differs from the fingerprint of the test tool. This random value is called fp_rand.
2. Configure the test tool to send shippairing requests with the following changes:
   a. The value of the key "trustPar" is set to the value of fp_rand.
   b. The value of the key "digest" is calculated using the value of fp_rand instead of the value of the test tool's fingerprint (i.e. instead of the original value of "trustPar").
3. Configure the test tool to repeat unsuccessful SHIP connection attempts latest within 60s.
4. Enable the SHIP and SHIP Pairing Service functionality of the test tool.
5. Configure the test tool for SHIP and shippairing with the DUT.
6. Monitor the test tool's SHIP connection state for 5 minutes.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Expected result: During the last monitoring phase, no SHIP connection is established between DUT and test tool.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

5 Parallel test cases

Test cases defined in this chapter monitor a DUT's behaviour parallel to the execution of test cases of chapter 4. This means that each applicable parallel test case begins before the execution of test cases of chapter 4, and ends latest after all test cases of chapter 4 have been passed successfully.

5.1 TC_SPS_PARA_001: No shippairing service

Test Case ID: TC_SPS_PARA_001

Roles: DUT = devA. Test tool = monitoring.

Reference: [SHIPPairingService], section 4.1

Description: This test verifies that devA does not announce a shippairing service.

Test applicability: This test case applies if the DUT has the role devA and the vendor filled PAR_spsSvcByDevA with "no".

Preconditions:

- None.

Test steps:

1. Monitor for shippairing services of the DUT*[1].

Expected result: The DUT does not announce a shippairing service.

*[1]: There are several technical possibilities to verify if a shippairing service stems from the DUT or another device. This test case does not require the verification with a particular methodology. Rather, a non-exhaustive list of possibilities is described. It is the responsibility of the tester to realise the verification with one of these possibilities or with a suitable alternative methodology:

1. Example: Perform tests in an isolated network with the DUT and the test tool as the only devices. Every shippairing service not announced by the test tool must then be from the DUT.
2. Example: Let the test tool analyse all shippairing requests that are not created by the test tool itself: there must be no TXT record where the key "trustId" is set to the SHIP ID of the DUT.
3. Example: Determine the MAC address of the DUT and verify if an mDNS message has been sent from this MAC address.

5.2 TC_SPS_PARA_002: Only addCu-requests

Test Case ID: TC_SPS_PARA_002

Roles: DUT = devZ. Test tool = monitoring.

Reference: [SHIPPairingService], section 4.1

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Description: This test verifies that devZ does not announce shippairing services with the TXT record key "type" having a different value than "addCu".

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- PAR_typeRestrDevZ must be set to "yes".

Test steps:

1. Monitor for shippairing services of the DUT.

Expected result: Each shippairing service of the DUT has a TXT record key "type" with the value "addCu".

5.3 TC_SPS_PARA_003: trustNonce variation

Test Case ID: TC_SPS_PARA_003

Roles: DUT = devZ. Test tool = monitoring.

Reference: [SHIPPairingService], section 6.2

Description: This test verifies that devZ does not create predictable values for trustNonce*[1].

Test applicability: This test case applies if the DUT has the role devZ.

Preconditions:

- None.

Test steps:

1. Log all shippairing services of the DUT in the order of their occurrence.

Expected result: The ordered sequence of all "trustNonce" values of the shippairing requests appears to be a sequence of random values. This means the sequence does not show any obvious pattern*[1,*2,].

*[1] Note: Subsequent versions of this test specification might explicitly define stronger test criteria to detect insufficient random number generations.

*[2]: In addition, a test tool might also apply a statistical analysis on the trustNonce values to support the tester in the rating of their randomness. However, in this version of the test specification it is sufficient that the tester assesses the randomness of the values by visual inspection of the ordered sequence of the "trustNonce" values. A non-exhautive list of example criteria is given:

- No value occurs twice or more.
- The sequence of the values does not appear to increase or decrease by a certain constant.
    - Counter-example: The sequence [E71...A1, E71...A3, E71...A5, ...] increases by 2 with every value.
- In the hexadecimal format, the sequence of the values does not appear to be rotated or shifted.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

- Counter-example: The sequence [E71...BA1, 1E71...BA, A1E71...B, ...] rotates to the right by one hexadecimal digit with each value.
- Counter-example: The sequence [E71...BA1, 71...BA15, 1...BA153, ...] shifts to the left by one hexadecimal digit (and adds a new digit at the right) with each value.

5.4 TC_SPS_PARA_004: No connection attempt to shippairing port

Test Case ID: TC_SPS_PARA_004

Roles: DUT = devA. Test tool = monitoring.

Reference: [SHIPPairingService], section 5.3

Description: This test verifies that devA adheres to the rule that the port specified in the shippairing SRV record is not used for data transmission or connection initiation, as the port is currently unused/reserved for future use.

Test applicability: This test case applies if the DUT has the role devA.

Preconditions:

- None.

Test steps:

1. Whenever another test case has a SHIP instance of the test tool with the role devZ, the test tool monitors the respective shippairing-SRV port*[1] for any incoming SYN/connection attempts or data packets from the DUT.

*[1] Note: This should not be confused with the SHIP port, which must be different from a shippairing port.

Expected result: The DUT (devA) SHALL NOT initiate a connection attempt or send any data to the port specified in the shippairing SRV record.

5.5 TC_SPS_PARA_005: No parallel shipparing requests to the same devA

Test Case ID: TC_SPS_PARA_005

Roles: DUT = devZ. Test tool = monitoring.

Reference: [SHIPPairingService], section 5.5

Description: This test verifies that devZ does not create parallel shippairing requests to the same devA*[1].

*[1] Note: Test case TC_SPS_DISC_009 creates a situation where the DUT (devZ) creates several shippairing requests to the same devA. Test case TC_SPS_PARA_005 monitors if some of the requests are present at the same time.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.

Test applicability: This test case applies if the DUT has the role devZ and another test case of chapter 4 with a devZ as DUT and a single devA SHIP instance of the test tool is executed*[2].

*[2]: This means that TC_SPS_PARA_005 is executed in parallel of a suitable other test case (esp. TC_SPS_DISC_009). In that other test case, the test tool has the role devA and a single SHIP instance. The devA SHIP ID must be known for the evaluation in TC_SPS_PARA_005.

Preconditions:

- None.

Test steps:

1. Log all shippairing services and their goodbye messages (deletion) of the DUT in the order of their occurrence. Log the messages together with a timestamp.

Expected result: The log does not show that two or more shippairing requests for the devA SHIP instance*[3] are valid at the same time.

*[3]: The verification affects shippairing requests with the key "forID" set to the SHIP ID of devA as described above.

Copyright © 2026 EEBus Initiative e.V. All rights reserved.